



17

PACKERS AND UNPACKING MALWARE



Modern malware needs built-in protections to evade modern endpoint and network defenses. Ideally, these protections will also impede reverse-engineering and help protect the malware's payload and internals from investigators. One option is a *packer*, a tool that adds obfuscation and protection to software. Many of the malware samples you're likely to encounter in the wild will be packed, so it's very important to be familiar with them. This chapter will introduce various types of malware packers, their architecture, how they work, and, most importantly, how to bypass them in order to access the malicious code they contain.

Types of Packers

When packers were originally designed, there was nothing inherently malicious about them. They were used simply to compress files such as executables. Once malware started using packing programs, however, the word *packer* became synonymous with malware.

There are multiple varieties of packers. The most common type, which I'll call *general packers*, is often free, open source, or otherwise widely available. Examples of general packers include NSPack, MPRESS, and UPX (Ultimate Packer for Executables), which are freely available to anyone, including malware authors. The downside of malware using general packers (for their authors, at least) is that they're typically very easy to unpack using automated tools or manual analysis since they weren't designed with anti-analysis in mind. Still, some malware samples in the wild use these simple packers as a quick and inexpensive way to provide basic obfuscation for their payloads.

The second type of packer includes *commercial packers* such as VMProtect, Themida, and Armadillo. Also known as protectors or obfuscators, these were created primarily for protecting the intellectual property of legitimate software. Because a lot of research went into their design, commercial packers are often very difficult to unpack. Luckily for us, they're not as commonly used to protect malware.

Finally, the last type of packer is designed specifically for malware. These packers, sometimes called *crypters*, are often developed by threat actor groups or affiliates and sold on hacker forums. Some examples of this type of packer are Warzone, Atilla, and Softinca Crypter, the last of which is shown in **Figure 17-1**.

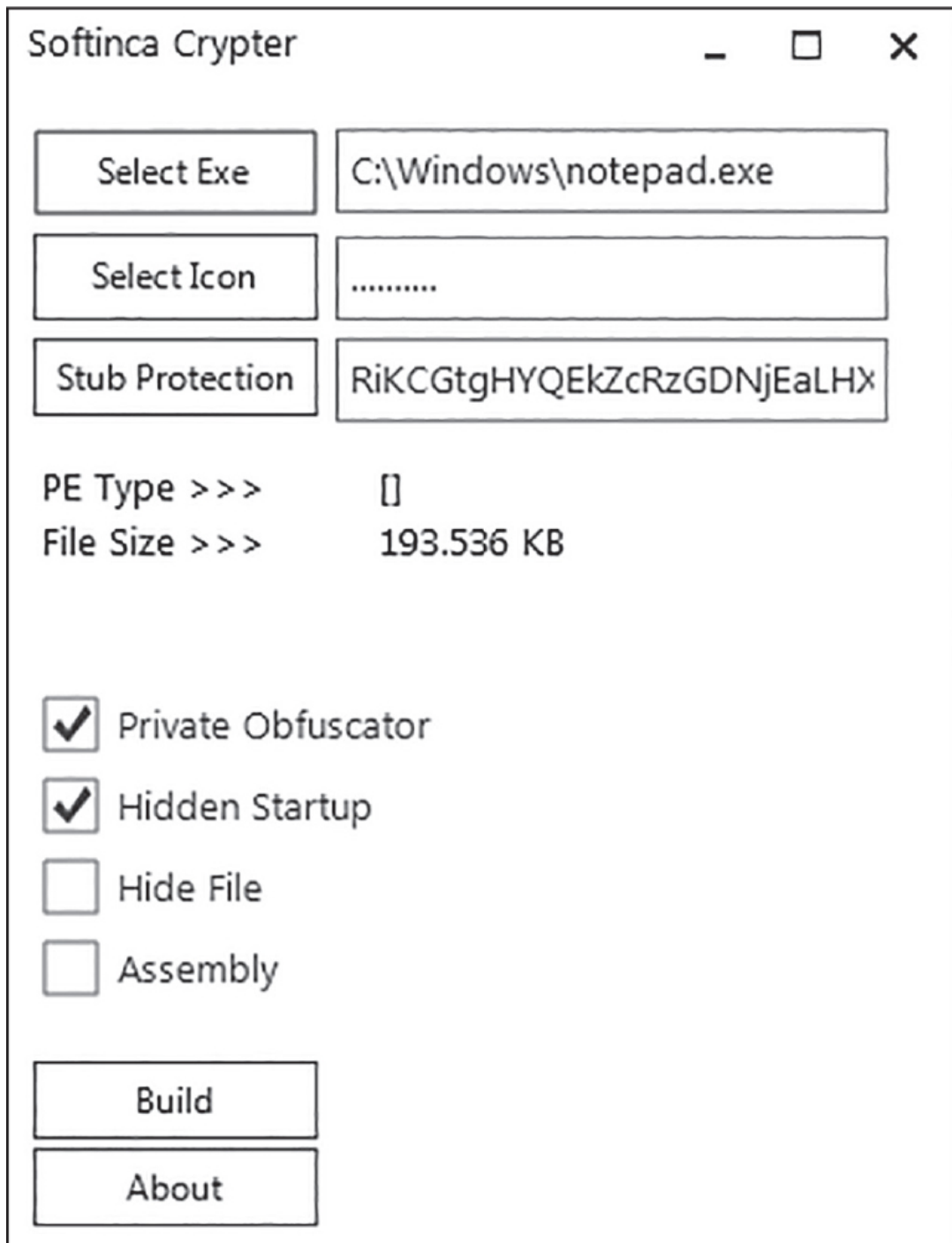


Figure 17-1: The interface for Softinca Crypter

Softinca Crypter takes an executable file (the unpacked malware payload) as input, adds protection such as code obfuscation and the ability to hide the payload execution from the victim, and then creates the packed executable file.

NOTE

Since packer is often used as a catch-all term for protectors, obfuscators, and crypters, I'll follow that convention throughout this chapter.

Packer Architecture and Functionality

When a malware executable is run through a packing program, the program encrypts and compresses the PE's sections (*.text*, *.data*, *.rdata*, *.rsrc*, and so on) within the executable. The packer also adds an *unpacking stub*, a (usually) small section of code that's responsible for decrypting the executable's sections once it's run on the target host, to the packed executable. **Figure 17-2** illustrates this process.

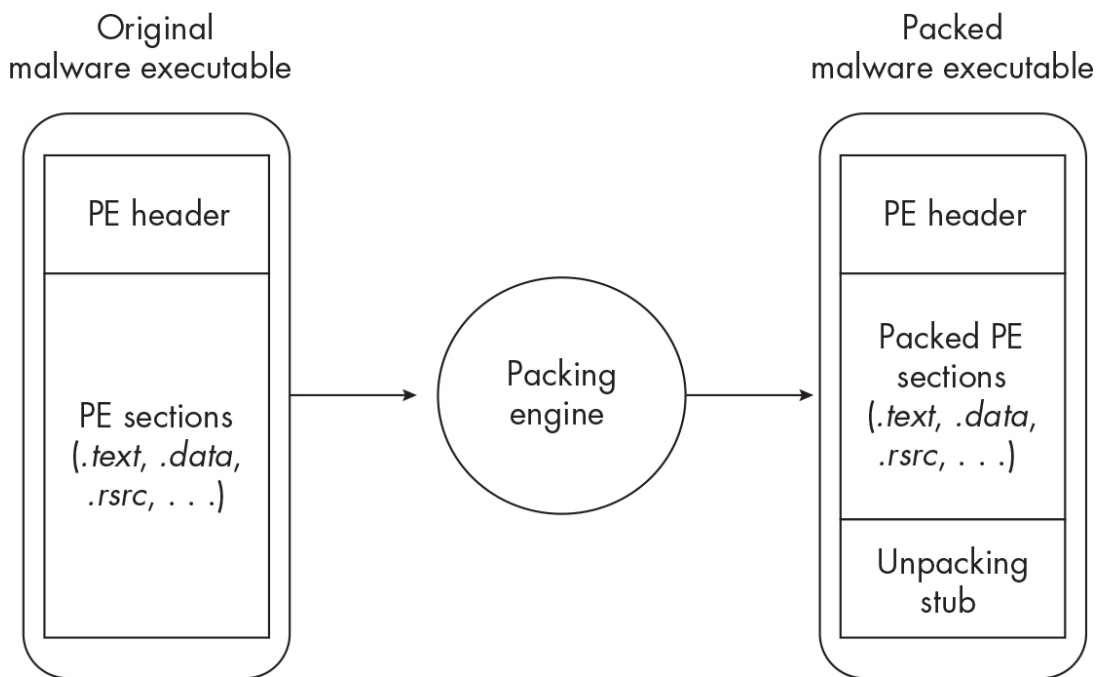


Figure 17-2: The malware packing process

You can see in **Figure 17-2** that the unpacked malware executable is being run through the packing program, which takes its code and data and encrypts (or packs) it. The unpacking stub is also added to the packed file. Upon running on the victim host, the unpacking stub decrypts the packed code and data, loads the unpacked payload into memory, and executes the payload by transferring control flow to the *Original Entry Point (OEP)*, as shown in **Figure 17-3**.

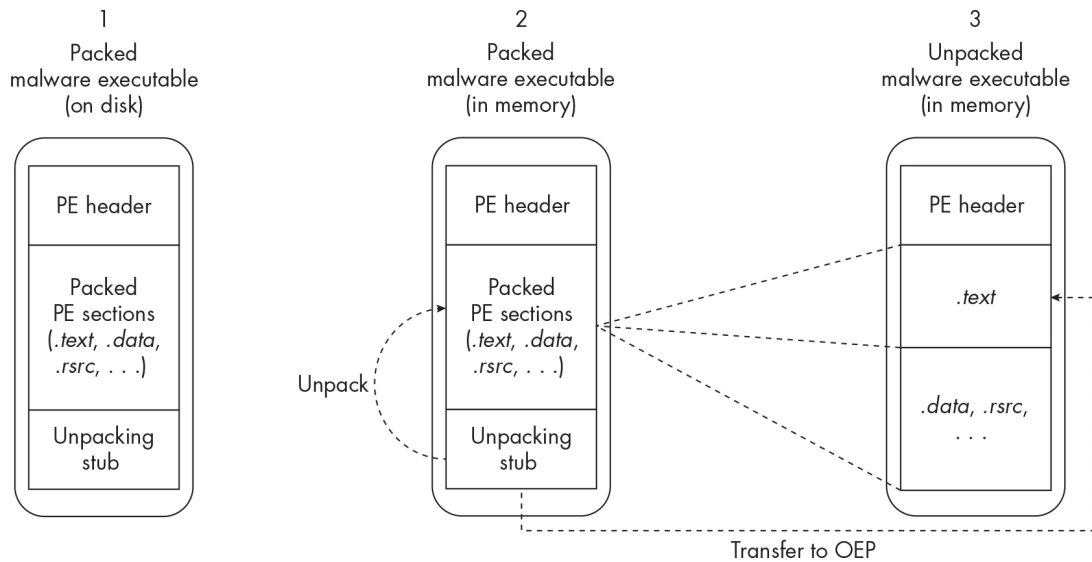


Figure 17-3: Packed malware being unpacked into memory

Let's take a look at this process in more depth.

Unpacking the Malware Payload

Before the malware's payload can be executed, it must be unpacked. The unpacking stub must decrypt (or deobfuscate) the original executable file's code and data into memory. For most Windows executables, this will involve running decryption and decompression algorithms on the packed executable, allocating memory space using Windows API functions such as `VirtualAlloc`, and writing the newly unpacked executable to the new memory region.

Unpacking can occur in one or more stages. General packers, such as UPX, simply unpack the executable in memory and run it. Other packers, especially custom packers designed for malware, may have more than one unpacking stage, as shown in [Figure 17-4](#).

Packed malware executable (in memory)

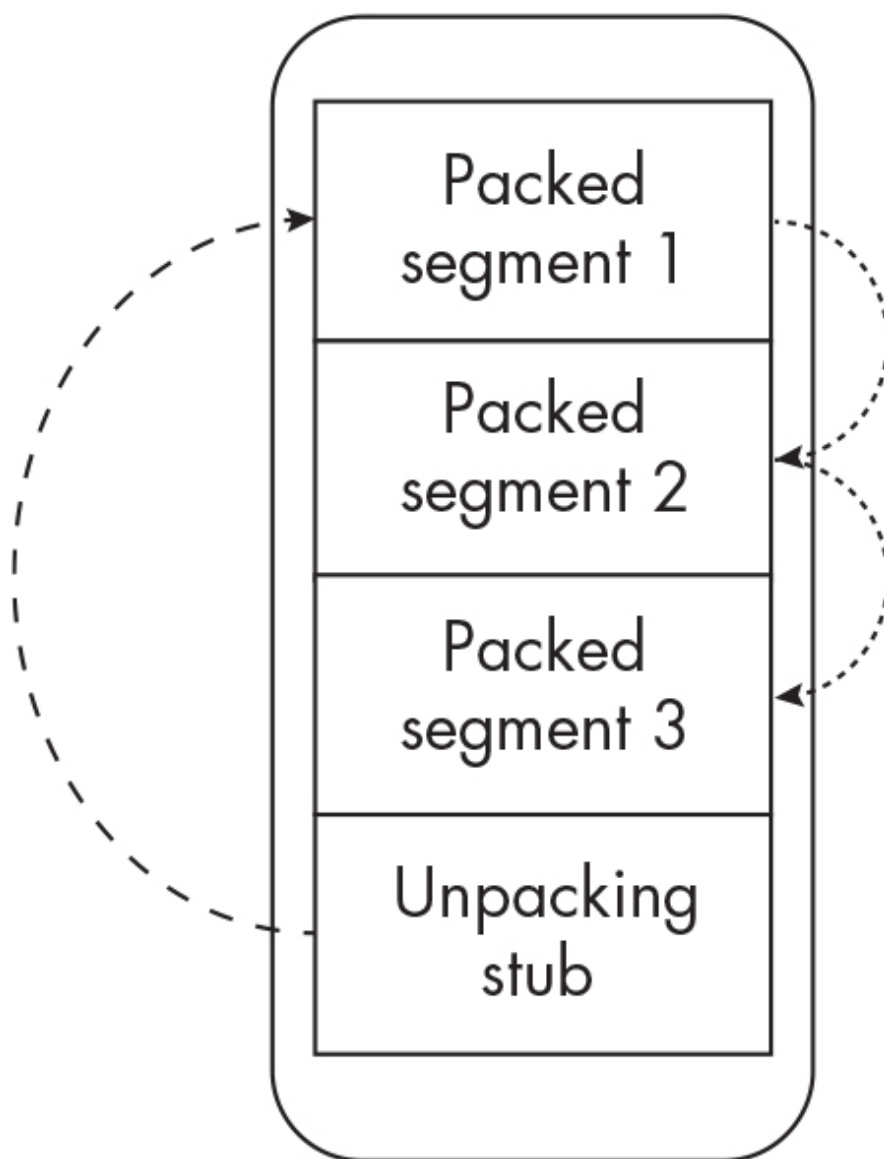


Figure 17-4: Malware using multistage unpacking

The simplified process shown in **Figure 17-4** illustrates a packed malware sample unpacking code Segment 1, which unpacks code Segment 2, which in turn unpacks code Segment 3. By unpacking the code in pieces, malware can evade host-based defenses that are looking for the entire malicious code in memory. This sort of unpacking routine also complicates the malware analysis process since the analyst will have a more difficult time understanding the unpacking process and identify-

ing all the locations in memory into which the malware is being unpacked.

Resolving Imports

Once the malware's payload has been unpacked into memory, the unpacking stub must resolve the imports of the original executable. Remember, imports are the Windows libraries (DLLs) that allow the executable to function within a Windows environment. When a malware sample is run through a packing program, the import address table (IAT) is typically obfuscated or hidden to mask the intent of the program and better evade defenses. This IAT must be rebuilt for the malware's unpacked executable to function as originally intended.

It's common for packed malware to contain only a few entries in its IAT, including the `LoadLibrary` and `GetProcAddress` functions. These two functions are often used to load additional libraries, resolve function addresses, and rebuild the IAT of the original executable. The `LoadLibrary` function loads each library required by the original malware executable, and the `GetProcAddress` function gets the address of each required function. You should be suspicious if you come across an executable file with only a few libraries listed in its IAT.

Packers might also remove all imports, leaving behind an empty IAT. This is the stealthiest approach, but the unpacking stub is then required to do the heavy lifting and resolve all imports. It starts by getting the address of the `LoadLibrary` and `GetProcAddress` functions, and then it loads each library and resolves each function address needed for the unpacked payload's operation.

Alternatively, certain packers will not rebuild the original IAT at all. In this case, the imports and address resolution process must be completely handled by the unpacked malware itself. If this is the case, you will see the malware likely using `LoadLibrary` and `GetProcAddress` to resolve its imports *after* the unpacking process has completed.

Transferring Execution to the OEP

Finally, once the malware is unpacked and the IAT is rebuilt, the unpacking stub must transfer execution from its own code to the OEP of the executable. The OEP is where the unpacked malware payload will begin executing its code. This transfer of execution, often called the *tail jump* or *tailing jump*, usually appears in the form of a jump (`jmp`), return (`ret`), or `call` instruction at the end of the unpacking stub. Once this instruction is hit, the program control will be transferred to the unpacked code in memory, and the unpacked malware payload will finally run.

Many packers, especially ones designed specifically for malware packing, implement some form of code injection technique to try to bypass defenses and hide on the host. For example, a packer might allocate memory inside an arbitrary or specific process on the victim host, write its unpacked code into that memory, and transfer execution to this code. In this case, you'll likely spot some of the process injection techniques and related functions covered in [Chapter 12](#). This is an important point to remember, and we'll come back to it later in the chapter.

Next, let's take a look at how to identify whether malware is packed.

How to Identify Packed Malware

Before you begin to unpack a malware sample, you have to identify whether it's even packed to begin with. There are a few ways to do so.

Viewing Imports

One of the simplest and most effective ways of determining whether a sample is packed is to inspect the file's imports. You can do this with almost any PE file viewer tool, such as CFF Explorer, PEStudio, and PE-bear. Packed malware may have only a few imported libraries and functions. The screenshot from PEStudio in [Figure 17-5](#) illustrates what packed malware might look like.

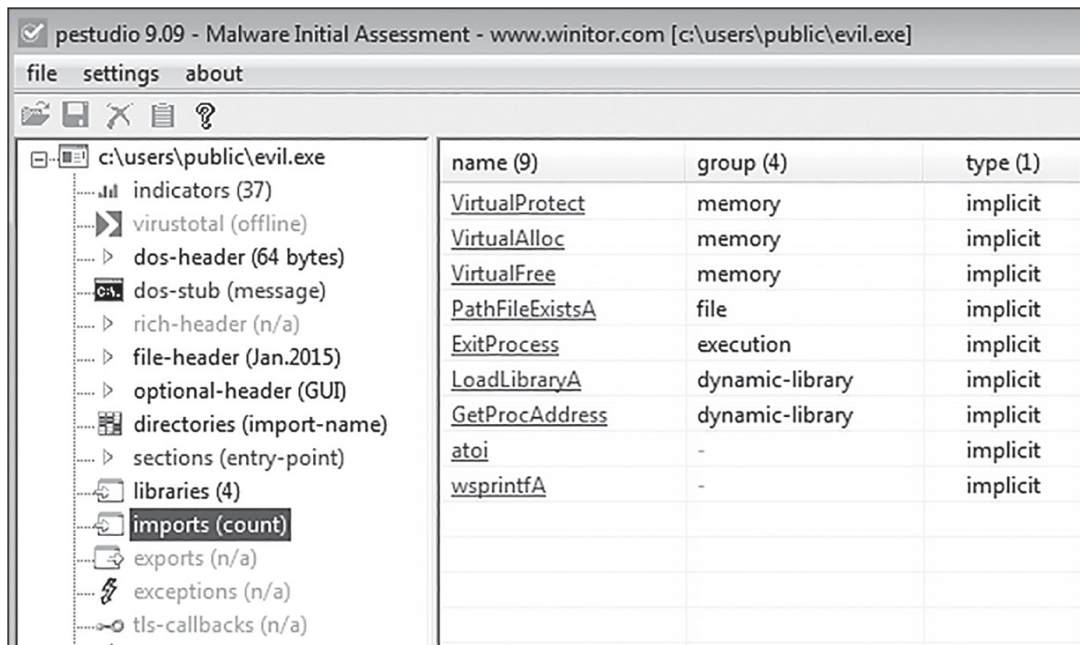


Figure 17-5: Viewing a packed malware sample in PESTudio

Notice how there's only a limited subset of functions listed, two of which are `LoadLibraryA` and `GetProcAddress`. By comparison, in [Figure 17-6](#) you can see an unpacked malware sample's list of imports.

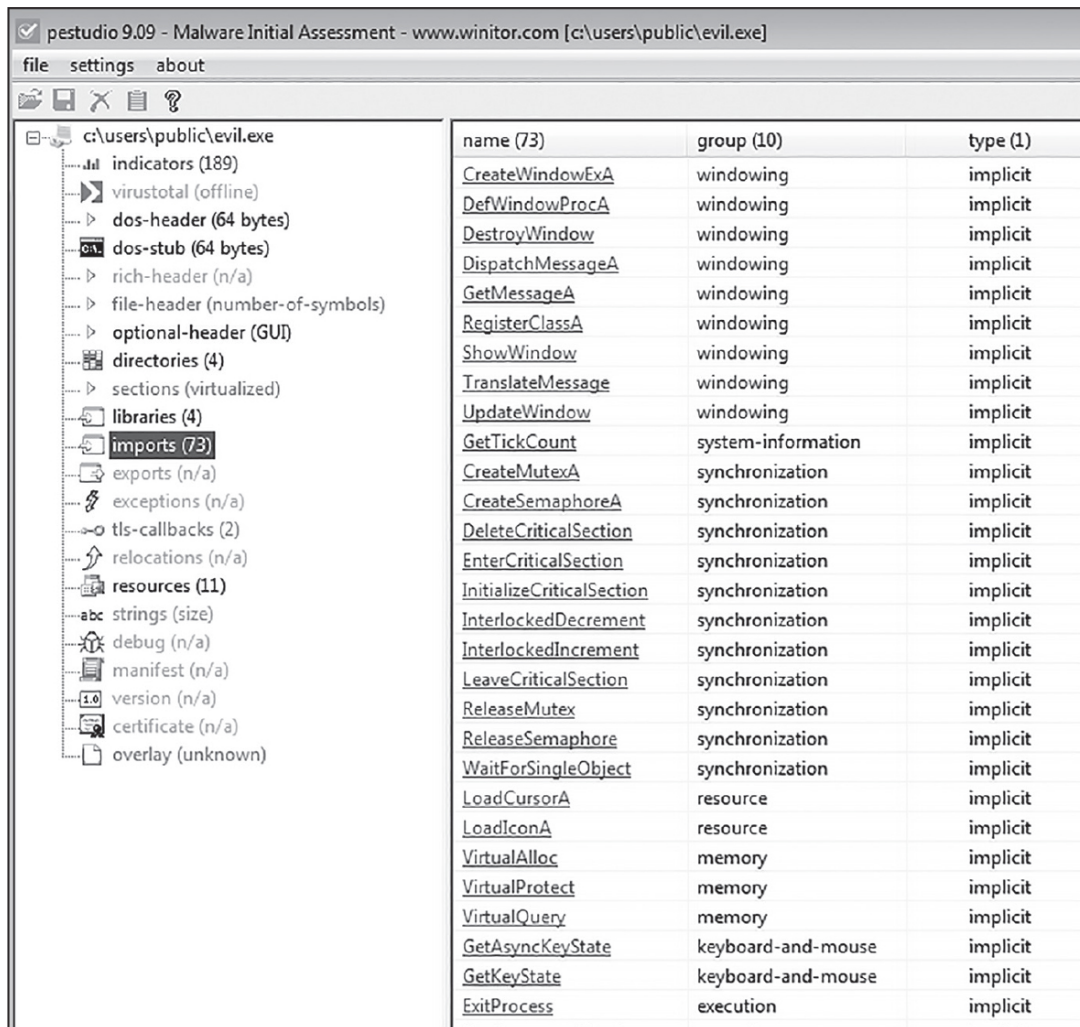


Figure 17-6: Viewing an unpacked malware sample in PESTudio

Clearly, the malware sample in **Figure 17-6** has more imports. Again, if malware has only a limited list of imports, there's a good chance that it is packed.

Inspecting Strings

Another good way to determine whether a sample is packed is to inspect its strings. Packed malware will have either many strings that aren't human readable or hardly any strings at all. This is because during the packing process, the packer compresses, encrypts, or otherwise obfuscates the data in the file to make analysis more difficult and to bypass defenses. The screenshot from PESTudio in **Figure 17-7** shows what packed malware's strings might look like.

type (2)	size (bytes)	file-offset	blac...	hint (332)	g	value (4931)
ascii	1500	0x00021B64	-	size	-	WrSzXvXtjofnYJiXs/eXnYyCLo/JAzcQbAiWxymAaP1inC6QPcwwh1LYGLF0Y8YNLidU6njlkj0l...
ascii	1500	0x00022144	-	size	-	yAdv8Tjr5AGgyMR4Xb804++hH+C0SQFQ78Xylj0c85E+Mp2FHFENsbkAysT0vTPbp6DgUg...
ascii	1500	0x00022724	-	size	-	BjeYdzq09jly3m3j+FDzuGo5Fzue6yYfJG6ts14TXltM1/UYKbwtm7Anq/pABhqWAsHsahx6h/...
ascii	1500	0x00022D04	-	size	-	zm8RvcNn9137TyG16VPuopWZ00NRIQjm3pFIqeaValNjTDpKuQ7lInvtO4Zp6lOfZCv8TlIwvx...
ascii	1500	0x000232E4	-	size	-	kThhTfeDL58kfM4rYU63T42NREBz5Zl84a7tFotH4LmNlL+6ludq/Gh+f0ngQ7XzcZ7MCovJCA...
ascii	1500	0x000238C4	-	size	-	dxMKZyLpr0T0RGDEknG9u1+VtUertCl5m4nf0sfKe+size+QEz88ITIEhCVF/HZc3plxygulep9...
ascii	1500	0x00023EA4	-	size	-	tgG/LYo6LoUGyyuemWgPOGvMaOusBjsk1WvROlwV6jIhkl5byeHKZ4JBHJ5hSuhOzOrVrixCj...
ascii	1500	0x00024484	-	size	-	htnbv6pskNTLIHT7w8On/FBMMzcyIvfzhcMP318iJW7R632WR35xuEqoH87KD9q+LbI83U1...
ascii	1500	0x00024A64	-	size	-	3vZefgAM6dTT8UQgnx10SedU8rc9SK3DkOq4FThYMxkbCRNnCLlw8wV4bxQ0ysOvwL/L6sL...

Figure 17-7: Strings from a packed malware sample

In contrast, an unpacked sample should have many cleartext (deobfuscated) strings.

Calculating the Entropy Value

Packed malware will likely have a high entropy value. *Entropy* is the measurement of randomness in data. High entropy indicates that the data may be encrypted or compressed, and in the context of packing, it suggests that a sample is likely packed. The maximum possible entropy value is 8; the closer the file gets to this value, the more likely that it is packed.

There are many tools that can calculate the entropy of a malware executable, but I've used PEStudio as an example once again. **Figure 17-8** illustrates the entropy value of a packed malware sample.

property	value
md5	<u>DCC0D02B0936131D89752DAF8CCA3B5C</u>
sha1	<u>195C29CFBD04F7BF2E2C90C930F34C712F436883</u>
sha256	<u>24D775CDE5E58069948E25D7E38BA2BC41326E5A06EF33C653B958956CE8BABA6</u>
md5-without-overlay	wait...
sha1-without-overlay	wait...
sha256-without-overlay	wait...
first-bytes-hex	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 B8 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
first-bytes-text	M Z @
file-size	592384 (bytes)
size-without-overlay	wait...
entropy	6.993
imphash	n/a
signature	<u>Microsoft Visual C++ v6.0</u>
entry-point	55 8B EC 6A FF 68 08 C9 44 00 68 80 75 43 00 64 A1 00 00 00 00 50 64 89 25 00 00 00 00 83 EC 58 53

Figure 17-8: Viewing a packed malware entropy value in PEStudio

A good rule of thumb is that a packed executable file will have an entropy value of about 6 or higher. Anything lower than this, and the

chances that the file is packed decrease.

NOTE

Keep in mind that some non-PE files (such as document files) will always have a high entropy value, so the “6 or higher” rule of thumb applies only to PE files.

Checking PE Sections

You can also use the PE file section information to determine whether malware is packed. As you may remember from [Chapter 1](#), executable files have multiple sections called `.text`, `.data`, `.rdata`, `.rsrc`, and so on. In a normal, nonpacked executable, the sections will be labeled with these names. Sometimes packers rename these sections in the file in a way that identifies the packer. For example, the UPX packer renames them to `UPX0`, `UPX1`, and so on. Another anomaly of packed malware is that it often has too many or not enough sections. A normal, nonpacked executable typically has four sections (give or take a few), so a file with nine sections or only one or two sections is a possible red flag and should be investigated further.

There are many different tools that allow you to view PE file section information. In [Figure 17-9](#), I’ve used PE-bear to show what a packed malware sample may look like.

Name	Raw Addr.	Raw size	Virtual Addr.	Virtual Size	Characteristics	Ptr to Reloc.	Num. of Reloc.
▷	1000	27000	1000	27000	E0000060	0	0
▷ petite	28000	1000	28000	1000	40000040	0	0

Figure 17-9: Viewing a packed malware sample’s PE sections in PE-bear

The file in this figure has only two sections. One is completely unlabeled, and the other is called *petite*, which is a nonstandard executable section name. This file is likely packed, possibly using the Petite packer.

Lastly, a PE file's section sizes can be another great indicator of packing. Each section has two size characteristics: the raw size and the virtual size. The PE file's *raw size* is the size of the section on disk, and the *virtual size* is the size once it is executed and subsequently mapped to memory. If you spot a malware sample that has a raw size of zero and a virtual size of nonzero, this is a good indication that the malware is packed. In this case, the malware may be trying to hide its code in another section, for example.

Using Automated Packer Detection

Finally, the simplest method of determining whether or not a malware sample is packed is using an automated packer detection tool. There are several of these tools available, but my personal favorites are Detect It Easy (DIE), Exeinfo PE, PE Detective, and CFF Explorer. These tools provide information such as the file's entropy, section names and sizes, compiler data, and sometimes (in the best case) the name of the packer itself.

Exeinfo PE, for example, attempts to identify the packer variant using static signatures. In **Figure 17-10**, you can see Exeinfo PE in action: it has identified that this malware sample is possibly written in .NET and potentially packed with the DeepSea obfuscator.

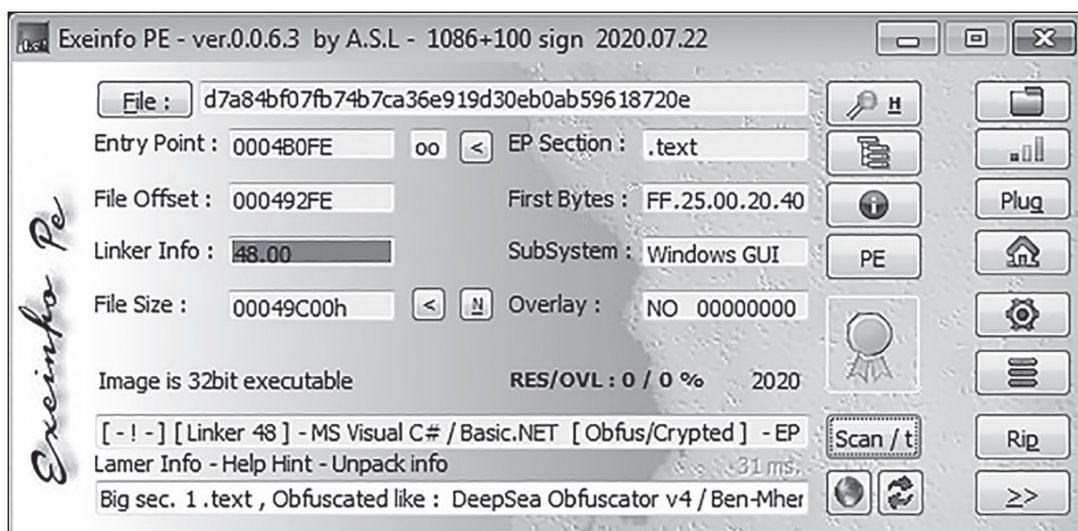


Figure 17-10: Identifying a malware's packer in Exeinfo PE

While automated packer detectors like Exeinfo PE are not always 100 percent accurate, using them is a great first step in inspecting malware executable files, and they can provide vital tips to guide your analysis and unpacking process. It's always best to try a few of these tools and see which provides the best output for the malware sample you are inspecting.

NOTE

Many of these tools update their detection databases regularly, so be sure to install any updates for these tools to ensure that you're getting accurate results.

Automated Unpacking

Once you've identified that the malware sample is indeed packed, you can start thinking of ways to unpack it. You may be asking yourself, *Why should I unpack the malware sample? Can't I simply run it and analyze it as is?* It's always an option to simply run the malware in a sandbox or debugger and avoid unpacking entirely; in fact, I'll specifically address that in "Analyzing Without Unpacking" on [page 383](#). However, unpacking allows you to get to the heart of the malware and extract its payload, which is often necessary for you to fully understand the malware's capabilities and statically analyze its code.

While you can still understand a malware's behaviors without unpacking it, you may lose some nuance. An example is a malware sample that has hidden capabilities or acts differently depending on its analysis environment. In the context of evasive malware, for example, this could mean that the malware behaves differently in an automated malware sandbox. Without unpacking this malware and closely analyzing its code, you might miss some key behaviors, capabilities, and indicators. There are multiple ways to unpack a malware sample, including fully automated unpacking, sandbox-assisted unpacking, manual dynamic unpacking, and static unpacking. We'll begin in this section with fully

automated unpacking and sandbox-assisted unpacking, then dig into the other methods in the coming sections.

Fully Automated Unpacking

Fully automated unpacking is the simplest and fastest method of unpacking a malware sample, so it's always a good idea to try it first. Many general and common packers have either built-in capabilities for unpacking or automated unpackers written specifically for them. The UPX packing program contains a flag that allows files to be unpacked as well. Simply passing the `-d` parameter to UPX like so will unpack the file:

```
C:\> upx.exe -d file.exe
```

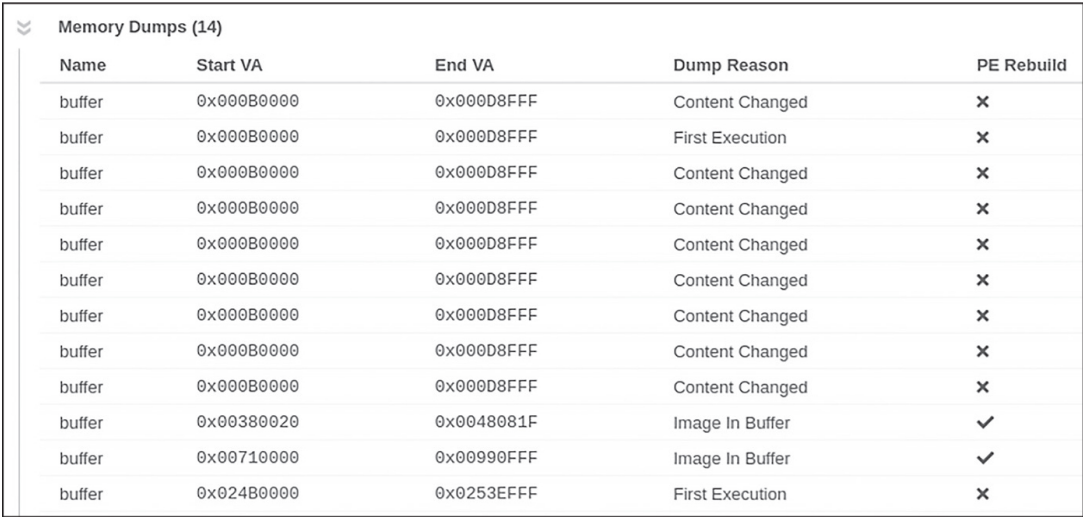
Other tools include Un{i}packer, which uses code emulation to unpack many common packers (such as MPRESS, ASPack, and, of course, UPX), and Universal Extractor 2 (UniExtract2), which can unpack many common packers and compressed file archives. You should first try to identify the packer being used; employ the techniques you've seen so far and then test out some of these helpful automated tools.

One important thing to remember is that common packers like the ones just mentioned can be modified by malware authors, since many of them are open source. It's relatively simple to modify these packers to prevent unpacking with these fully automated methods. Also keep in mind that advanced malware generally won't be packed (or, at least, won't be solely packed) with free and common packers, so make sure not to rely only on these automated tools. As you'll soon see, there are many other tools that can assist you in both automated and semiautomated unpacking.

Sandbox-Assisted Unpacking

The next-simplest way to unpack a sample is to use a malware analysis sandbox. Many malware sandboxes can automate malware unpacking,

typically by detecting malicious code in memory, hooking and monitoring key Windows functions often invoked during the malware-unpacking process, and automating the extraction of executable code from memory. One sandbox that does this fairly well is the commercial sandbox VMRay Analyzer. In the output shown in [Figure 17-11](#), you can see that VMRay Analyzer was able to extract the malware from memory in its various stages of unpacking.



Memory Dumps (14)				
Name	Start VA	End VA	Dump Reason	PE Rebuild
buffer	0x000B0000	0x000D8FFF	Content Changed	✗
buffer	0x000B0000	0x000D8FFF	First Execution	✗
buffer	0x000B0000	0x000D8FFF	Content Changed	✗
buffer	0x000B0000	0x000D8FFF	Content Changed	✗
buffer	0x000B0000	0x000D8FFF	Content Changed	✗
buffer	0x000B0000	0x000D8FFF	Content Changed	✗
buffer	0x000B0000	0x000D8FFF	Content Changed	✗
buffer	0x000B0000	0x000D8FFF	Content Changed	✗
buffer	0x000B0000	0x000D8FFF	Content Changed	✗
buffer	0x00380020	0x0048081F	Image In Buffer	✓
buffer	0x00710000	0x00990FFF	Image In Buffer	✓
buffer	0x024B0000	0x0253EFFF	First Execution	✗

Figure 17-11: Viewing malware executable code dumped from memory in the VMRay Analyzer sandbox

In this screenshot you can see how VMRay Analyzer has attempted to unpack the sample by dumping the malware’s memory at key stages of its behavior. For example, as you can see in the Dump Reason column, the malicious code is dumped from memory when its content is changed, when it is first executed, and when there is an executable file (image) in memory. VMRay Analyzer has also attempted to rebuild the PE headers of the dumped executable file so that it can be better analyzed in a disassembler or debugger.

There’s even a sandbox dedicated solely to unpacking: UnpacMe. According to its author, “UnpacMe automates the first step in your malware analysis process.” That is, it automates the unpacking process. UnpacMe is a commercial sandbox but (at the time of this writing) offers a free service with limited submissions per month. [Figure 17-12](#)

shows an example submission to UnpacMe and the resulting unpacked, downloadable payload files.

Results		
Submitted	Sample	Status
02/02/2021 13:19:37	59c7c188cb5c6e77646b917c6be75aa8ae3a32adcb566a21d4061f3a17e0f39	complete Unpacked!
Parent		
59c7c188cb5c6e77646b917c6be75aa8ae3a32adcb566a21d4061f3a17e0f39		Download
Unpacked Child		
59d0728f33e350c0e185e2ee7c17e7440e8b75672dd846379e827e437d62ae7f		Download
Unpacked Child		
a2a0bbbe8c9ee88441356010f63455c50dfcafe663db757bbc453c815e51d720		Download
Unpacked Child		
1fa7ef7be1c4343628e13635a51011631905dd6727ca166da53c9c9765ffed8c		win_formbook_g0 win_formbook_auto Download

Figure 17-12: A malware sample unpacked by UnpacMe

Sometimes malware sandboxes fail to successfully unpack the malware's payload. There are a number of reasons for this, such as evasive behaviors by the malware or failure to follow the malware's unpacking process in memory. Let's look at some manual unpacking techniques that will help you in the event that these automated techniques fail.

Manual Dynamic Unpacking

Manual unpacking involves identifying where the unpacking routine is located in the packed malware's code, determining how it is unpacking the malware, and following this unpacking process to "catch" the malware's payload in a freshly unpacked state. There are two forms of manual unpacking: dynamic and static.

Manual dynamic unpacking involves detonating the malware in a virtual machine environment and allowing the malware to execute and unpack as it normally would on a victim host, while using a debugger to follow the unpacking process and catch the unpacked payload in memory. By contrast, *static unpacking* involves reverse engineering the

malware's unpacking stub code, re-creating this code logic, and running it on the packed malware. This section will focus on dynamic packing, and we'll discuss static unpacking in "Manual Static Unpacking" on [page 382](#).

NOTE

Rather than providing very specific techniques for unpacking certain packers, I'll be covering more generic methods that you can use to dynamically unpack many variants of malware, regardless of the packer used. These techniques are not presented in any particular order; every packer behaves differently, so there's no one-size-fits-all technique. You may need to try different techniques or combine bits and pieces from several of them. This is what makes unpacking so challenging but also very rewarding!

The Quick-and-Dirty Option: Letting the Malware Do the Work

When packed malware runs on the victim host, it must unpack itself somewhere in memory. Detonating the malware in a virtual machine, letting the malware do the work of unpacking itself, and then extracting the unpacked code is one of the simplest forms of dynamic unpacking. You can attempt this first before delving into the more difficult unpacking techniques in this section.

Dumping a process from memory is fairly straightforward and can be done simply with Process Hacker or Process Explorer, as well as with other advanced task manager-type tools. In Process Hacker, you right-click the malware's running process and select **Create Dump File**, as shown in [Figure 17-13](#) with the *sample.exe* process.

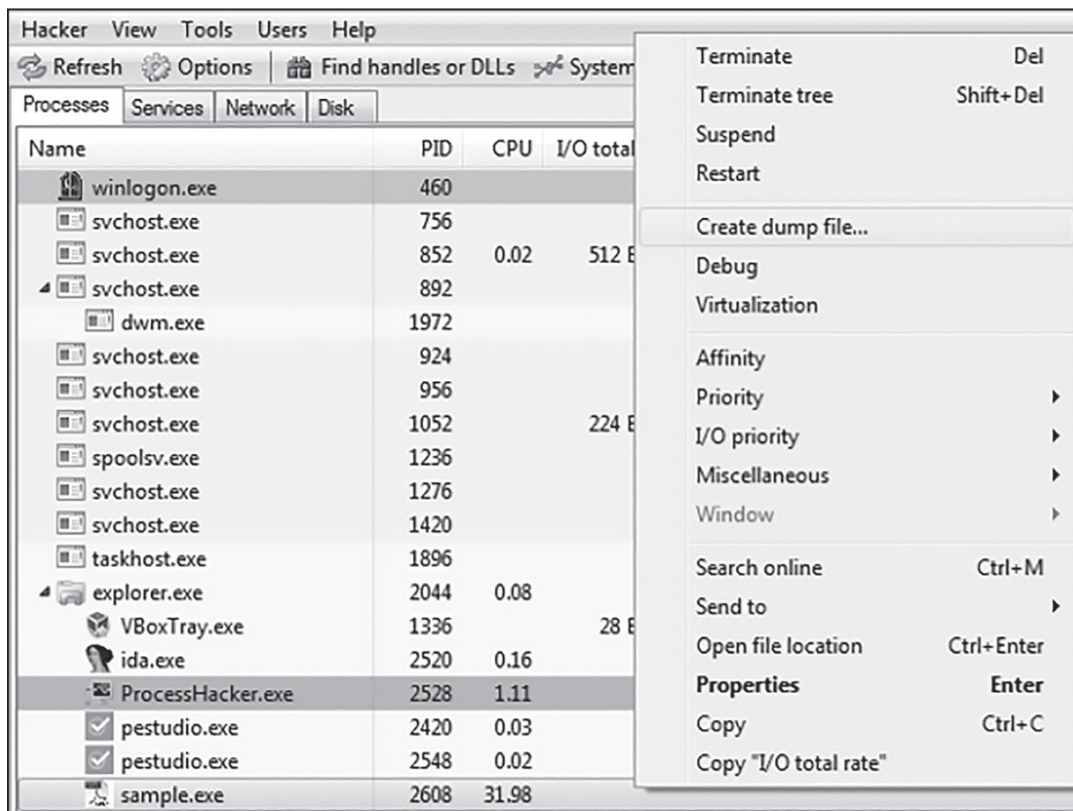


Figure 17-13: Extracting malware from memory using Process Hacker

This is a quick-and-dirty unpacking method, and it has some limitations. First, there's no way of knowing if the malware completely unpacked itself in memory yet. For example, evasive malware may detect the VM environment and refuse to unpack its payload. Not only that, but since this file was extracted directly from memory, it also hasn't been properly unmapped from memory and will be misaligned, which means that you likely won't be able to easily analyze it in a disassembler like IDA Pro. You can, however, inspect the strings of the file by running the Strings tool (or a tool such as PEStudio), which will give you hints about what this malware sample may be doing. You may even spot cleartext functions, C2 addresses, or decrypted data. There's a better option, however.

The tool Scylla allows you to extract this malware from memory and will automatically realign the file, fix the file headers, and even repair the IAT. Scylla is both a plug-in for x64dbg and a stand-alone tool, and it works exactly the same in both capacities. For example, say you have a malware sample, and after transferring it to your analysis VM, you det-

onate it. The sample may not immediately unpack itself into memory, so you choose to wait a minute or so to ensure the sample has completely unpacked itself. Next, you can run the Scylla tool, as shown in **Figure 17-14**.

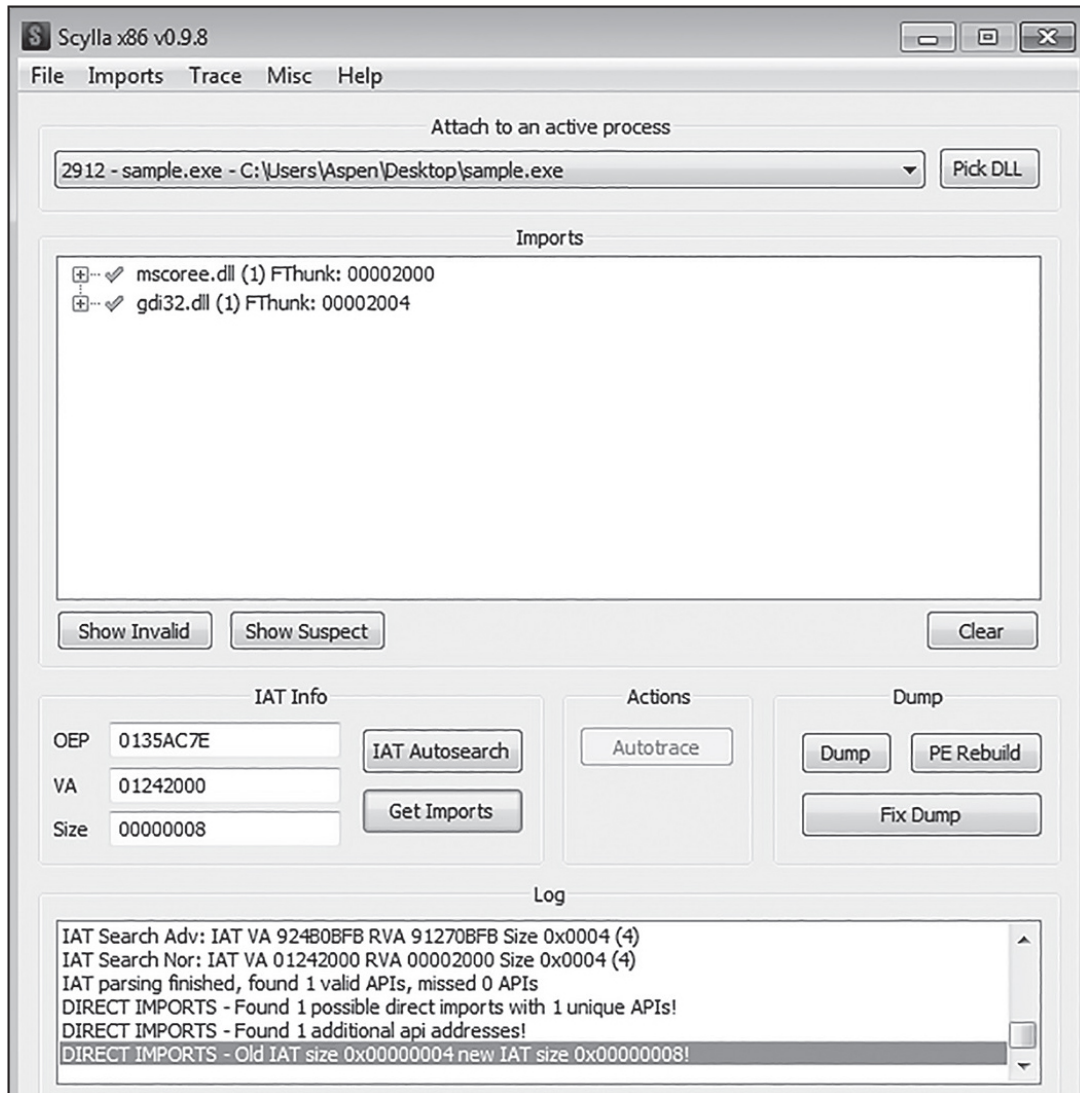


Figure 17-14: Malware process dumping with Scylla

Scylla allows you to select a target process in which to dump; this would normally be the malware's active process (*sample.exe* in this case). You can then click **IAT Autosearch** to automatically search the process memory for a possible IAT. Once the IAT has been found, click **Get Imports** to generate a list of imports that will populate the IAT once the process has been dumped. Next, clicking **Dump** will dump the process from memory to disk into an executable file, which essentially unmaps the process from memory.

Most advanced malware won't allow itself to be unpacked so cleanly. However, since this technique takes less than five minutes, it's always worth a shot. Now let's dig into some more advanced unpacking techniques.

Memory Operation Monitoring

Since the unpacking stub must allocate memory for the newly unpacked executable and modify its memory protections, we can assume that at some point, it will invoke Windows functions related to memory operations. The idea here is to set breakpoints in the debugger on these memory operation functions, run the malware, and closely monitor these operations, looking for an opportunity to dump the malware's unpacked code.

VirtualAlloc

`VirtualAlloc` is likely the most common memory allocation function you'll see, but `VirtualAllocEx` and `malloc` are also used. You can simply attach the packed malware sample to a debugger such as x64dbg, set a breakpoint on the memory allocation function you'd like to target (or set breakpoints on all of them), and run the malware sample. Once a memory allocation function breakpoint is hit, you must identify the base address of the newly created memory region and watch this memory region for new data. Let's see how this works in practice.

NOTE

To follow along with this example, you can find the required malware file on VirusTotal or MalShare at the following hash:

SHA256:

7b8fc6e62ef39770587a056af9709cb38f052aad5d815f808346494b7a3d00c5

Load the executable (which I've renamed in **Figure 17-15** to *badthing.exe*) into x64dbg and execute the **Run to User Code** function

in the Debug menu, which brings you to the entry point of the malware's code.

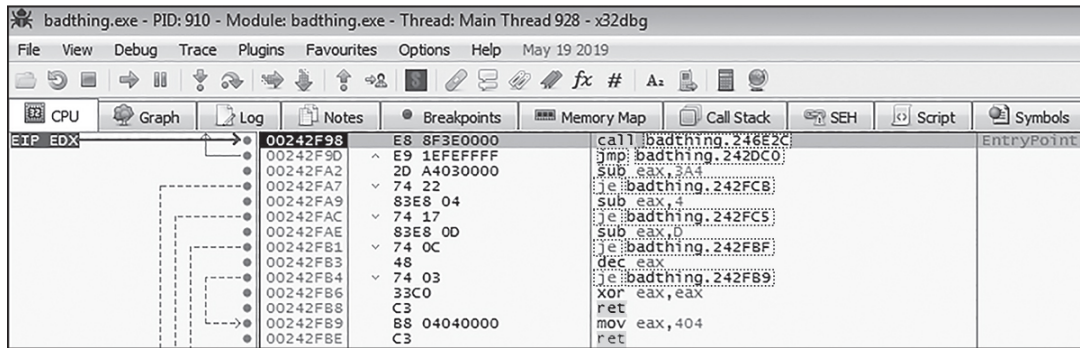


Figure 17-15: The malware's entry point in x64dbg

Next, place a breakpoint on the `VirtualAlloc` function (`bp VirtualAlloc`), as shown in [Figure 17-16](#). What we're hoping for here is that the unpacked executable will eventually be mapped to this region of memory.

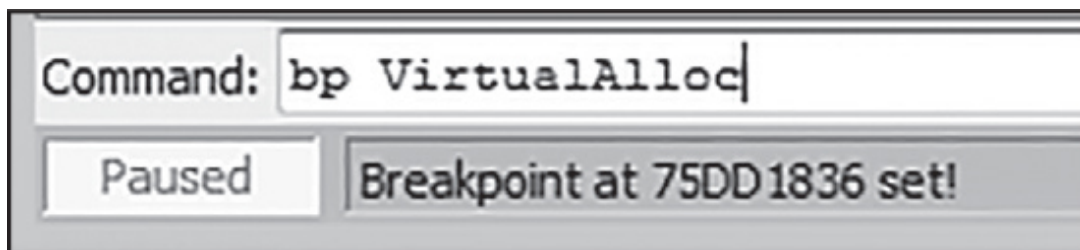


Figure 17-16: Setting a breakpoint on `VirtualAlloc`

Following this, continue to run the malware (by pressing F9) until this breakpoint is hit (see [Figure 17-17](#)).

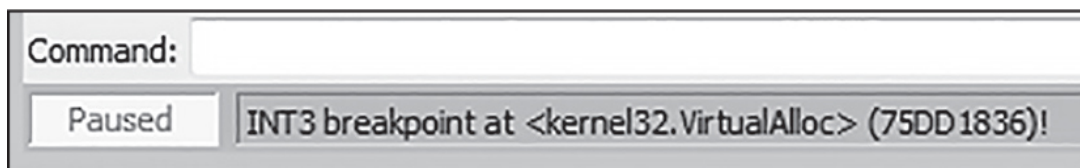


Figure 17-17: Hitting the breakpoint on `VirtualAlloc`

Once the breakpoint is hit, you can execute the malware until the function returns; select **Debug ▶ Execute Till Return** and then check the value of the EAX register, which contains the target memory region that was allocated by `VirtualAlloc` (see [Figure 17-18](#)).

EAX	000F0000	
EBX	00018000	
ECX	09FE0000	
EDX	0014DEC8	
EBP	004DF3A8	
<u>ESP</u>	004DF2F4	"r3%"
ESI	00017C9C	
EDI	11BE5298	

Figure 17-18: A newly allocated memory region in EAX

In my case, this memory region's base address is 000F0000. Right-click on this value in EAX and select **Follow in Dump**. You should see an empty memory region in the Dump window (see [Figure 17-19](#)).

Dump 1		Dump 2		Dump 3		Dump 4		Dump 5		Watch 1	[x=] Locals	Struct
Address	Hex									ASCII		
000F0000	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00			
000F0010	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00			
000F0020	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00			
000F0030	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00			
000F0040	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00			
000F0050	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00			
000F0060	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00			
000F0070	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00			
000F0080	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00			
000F0090	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00			
000F00A0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00			
000F00B0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00			
000F00C0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00			
000F00D0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00			

Figure 17-19: A fresh memory region in the x64dbg Dump window

If you continue to press F9 to run the code while watching this Dump window, you might see something of interest, such as an mZ header, which could be the malware's unpacked payload (see [Figure 17-20](#)).

Address	Hex								ASCII																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																		
000F0080	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00</

Figure 17-20: Viewing the unpacked code in allocated memory

You can extract this file from memory by selecting the start of the header (starting at the `m` in `mz`) and the rest of this memory region in the dump view, right-clicking, and selecting **Binary ▶ Save to File**. You can then inspect this file in a PE file viewer tool, looking for signs of successful unpacking, such as cleartext strings and imports.

Keep in mind that the `VirtualAlloc` and `VirtualAllocEx` functions subsequently call the lower-level API function `NtAllocateVirtualMemory`. Sneaky malware might invoke `NtAllocateVirtualMemory` directly instead of using `VirtualAlloc`. Setting a breakpoint on `NtAllocateVirtualMemory` can help in these situations.

HeapAlloc and Malloc

Malware will sometimes call `HeapAlloc` instead of `VirtualAlloc` when allocating memory. `HeapAlloc` differs from `VirtualAlloc` in two ways: it allocates memory on the program's heap rather than the stack, and it's a higher-level API call and will sometimes subsequently invoke `VirtualAlloc`. Because of this, it's more common to put a breakpoint on `VirtualAlloc`, but you can try setting a `HeapAlloc` breakpoint as well.

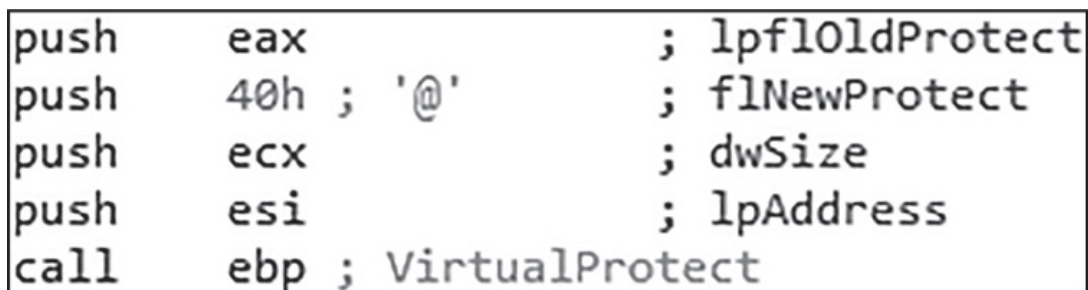
The C function `malloc` also allocates memory similarly to `HeapAlloc`. In fact, `malloc` typically invokes `HeapAlloc`, or in some cases, `VirtualAlloc`. It's not as common to see malware calling `malloc`, but it might if it's written in C or if it's trying to evade analysis or hide its activities, since `malloc` may get less attention from analysts. Setting a breakpoint on `malloc` can be useful in these scenarios.

Memory Deallocation

The `VirtualFree` (as well as its sibling, `VirtualFreeEx`) and `HeapFree` functions are used by Windows to release and free a memory region after it has been used. During the malware's unpacking process, once the malware allocates memory for its unpacked code and executes it, it will likely need to clean up afterward. Similarly to setting a debugger breakpoint on `VirtualAlloc` and `HeapAlloc`, setting a breakpoint on `VirtualFree`, `VirtualFreeEx`, or `HeapFree` can be a good tactic for catching unpacked malicious code before it has a chance to deallocate its memory. You may get lucky and spot an unpacked executable in one of these memory regions that can then be extracted for further analysis.

VirtualProtect

`VirtualProtect` and its sibling, `VirtualProtectEx`, are also functions that can be monitored for unpacked code. During the unpacking phase, after the malware allocates memory, it must set protections on the memory. The memory's protection represents what the malware is able to do with that memory region: write to it, read from it, execute it, or all of these. The memory protection option is passed to the `VirtualProtect` function as a parameter when the function is called, as shown in IDA Pro in [Figure 17-21](#).



```
push    eax                ; lpflOldProtect
push    40h ; '@'          ; flNewProtect
push    ecx                ; dwSize
push    esi                ; lpAddress
call    ebp ; VirtualProtect
```

Figure 17-21: Malware calling `VirtualProtect`, viewed in IDA Pro

In this screenshot, the `flNewProtect` parameter represents the new protection (in this case, `0x40`, which is `PAGE_EXECUTE_READWRITE`) that will be applied to the memory region (referenced in `lpAddress`). Also im-

portant is the `dwSize` parameter, which represents the size of the memory region that will have the new protection class.

Setting a breakpoint on `VirtualProtect` and `VirtualProtectEx` and watching for memory regions being marked as executable can help you catch malicious code that will soon be executed by the malware. Oftentimes, the malware's unpacked code resides in one of these memory regions. This area of code can then be dumped from memory for further analysis.

Table 17-1 lists some of the important memory protection constants.

Table 17-1: Memory Protection Constants

Constant (hex)	Constant value	Description
0x10	PAGE_EXECUTE	This memory region will now be executable only (writes and reads will result in an access violation error).
0x20	PAGE_EXECUTE_READ	This memory region will now be executable and readable, but not writable.
0x40	PAGE_EXECUTE_READWRITE	This memory region will now be executable, readable, and writable.

You can read about these and others in the “Memory Protection Constants” page in Microsoft’s documentation at <https://learn.microsoft.com/en-us/windows/win32/memory/memory-protection-constants>.

It's important to note that `VirtualProtect` and `VirtualProtectEx` subsequently call the lower-level API function `NtProtectVirtualMemory`.

Sometimes tricky malware samples may call `NtProtectVirtualMemory` directly, bypassing the normal `VirtualProtect` call, thus circumventing malware analysts who may only be on the lookout for `VirtualProtect`. In these cases, it can be helpful to simply put a breakpoint on `NtProtectVirtualMemory`.

When using breakpoints on `VirtualProtect`-like functions, it can be helpful to set hardware breakpoints on the memory region being modified rather than software breakpoints. As [Chapter 3](#) discussed, hardware breakpoints are more persistent, so evasive malware will have a tougher time removing them to circumvent analysis. I'll come back to this in "Hardware Breakpoints on Allocated Memory" on [page 365](#).

Memory Inspection During Runtime

Another useful general technique for unpacking malware is inspecting memory regions within the debugger, looking specifically for regions that are assigned executable protections. Such regions can indicate the presence of executable code. To do this in a debugger (in my case, `x64dbg`), select **Memory Map** near the top of the debugger window. The result should look similar to [Figure 17-22](#).

Address	Size	Info	Content	Type	Protection
00010000	00010000			MAP	-RW--
00020000	00007000			PRV	ERW--
00030000	00001000			PRV	-RW--
00040000	00001000			IMG	-R---
00050000	00039000	Reserved		PRV	
00089000	00007000			PRV	-RW-G
00090000	000F8000	Reserved		PRV	
00188000	00005000	Thread 850 Stack		PRV	-RW-G
00190000	00004000			MAP	-R---
001A0000	00001000			MAP	-R---
001B0000	00001000			PRV	-RW--
001C0000	00001000			PRV	-RW--
001D0000	00007000			MAP	-R---
001E0000	00002000			MAP	-RW--
001F0000	00001000	\Device\HarddiskVolume2\windows\		MAP	-R---
00200000	00001000			MAP	-RW--
00210000	00002000			MAP	-R---
00220000	00007000			PRV	-RW--
00227000	00079000	Reserved (00220000)		PRV	
002A0000	00001000			MAP	-R---
002B0000	00002000			MAP	-R---
002C0000	00039000	Reserved		PRV	
002F9000	00007000			PRV	-RW-G
00300000	00036000			PRV	-RW--
00336000	000CA000	Reserved (00300000)		PRV	
00400000	00001000	agent.exe		IMG	-R---
00401000	00007000	".text"	Executable code	IMG	ER---
00408000	00002000	".rdata"	Read-only initialized data	IMG	-R---
0040A000	00018000	".data"	Initialized data	IMG	-RW--
00425000	00008000	".ndata"		IMG	-RW--
0042D000	00001000	".rsrc"	Resources	IMG	-R---
00430000	00067000	\Device\HarddiskVolume2\windows\		MAP	-R---
004A0000	00001000			MAP	-R---
004B0000	00005000			PRV	ERW-G
004C0000	00027000	\Device\HarddiskVolume2\Users\Asp		MAP	-R---
004F0000	00001000			MAP	-RW--
00500000	00004000			PRV	-RW--

Figure 17-22: Viewing a memory map in x64dbg

This memory map shows a few memory regions that are marked as executable. You can sort this list simply by clicking on the **Protection** column header at the top. Once you spot one of these memory regions, you can dump it by right-clicking the memory address and selecting **Dump Memory to File**, or you can view it in the Dump window for closer inspection by selecting **Follow in Dump**. You can also view this memory region in the disassembler, which is helpful if you suspect the data contained in a memory region is code, for example. The debugger's disassembler will also help you quickly see the disassembled code. To do this, right-click the memory address and select **Follow in Disassembler**.

Some key things to look for are as follows:

Memory regions that have E (Executable) in the Protection column

This would indicate that there is executable code in this region. Regions with protection class ERW (Execute-Read-Write) should be prioritized; in many cases, malware that is about to execute its unpacked payload will assign ERW protection to this memory. Note that in some debuggers and memory editors, this protection class is called RWX (Read-Write-Executable).

Regions with the PRV memory type

You can spot this in the Type column. PRV is a great candidate for finding unpacked code.

Memory regions that have nothing in the Info and Content columns

If these fields are populated, it likely means that these memory regions are associated with the malware executable file itself (the packed version) and not its unpacked code. The memory region highlighted in [Figure 17-22](#) is a prime candidate for closer inspection.

Large data size (indicated by the Size column)

A substantial memory region (for example, 30,000 bytes) could indicate an unpacked executable in memory. This is not always true, however, as smaller memory sections can contain small bits of malicious code such as shellcode.

Any areas of memory that contain a PE header

It can be helpful to run a memory string scan for `MZ` or `This program`. To do this in x64dbg, right-click in the Memory Map area and select **Find Pattern**. Then, input your desired search string (such as `This program`) in the ASCII text box and click **OK**. This will search all memory regions for your chosen ASCII string. If you find a PE header in one of these regions, you should closely inspect it, as it could be the malware's unpacked payload (but take into account the next point).

Addresses below the 0x7 range

Memory addresses that start with `0x7` (`x77300000`, for example) are usually associated with legitimate Windows DLLs that are mapped into the malware's process address space, so you should focus less on this memory range. I say *usually* because malware can load malicious code into one of these regions, but it's not as common. Focus your efforts on the

addresses below the 0x7 range first. For 64-bit malware, this would be addresses below the range of 0x700000000000.

Hardware Breakpoints on Allocated Memory

As discussed in [Chapter 3](#), most modern debuggers offer both software and hardware breakpoint options. In most cases, software breakpoints will suffice (for example, for setting a breakpoint on a specific CPU instruction or API function call). However, hardware breakpoints are also useful for following a malware's unpacking process, as they can be set directly on memory regions. When a malware sample calls a memory allocation function such as `VirtualAlloc`, setting a hardware breakpoint on the newly allocated memory region can help catch the malware executing its unpacked code.

To do this in x64dbg, locate the memory region you're interested in (for example, a newly allocated memory region created via `VirtualAlloc`), view the memory in the Dump area, right-click, and select **Breakpoint ▶ Hardware, Write**. This effectively creates a persistent hardware breakpoint that will cause the malware program to pause when code or data is written to this memory region. Once code has been written to this region, you can set a **Hardware, Execute** breakpoint to catch the malware executing it. Note that memory breakpoints can also be used here for the same purpose.

Pitfalls of Following Memory Operations

Setting breakpoints on memory management functions can be an effective trick for following the malware unpacking process, but there are pitfalls. First, since these memory management functions are used often (both in legitimate and illegitimate circumstances), they can create a lot of noise. It's not uncommon to see thousands of memory management functions executed in a malware sample, so you may find that setting breakpoints on them will trigger constant pauses in your debugger execution, giving you too much to go through. In this case, coordinating

your efforts with process creations and code injection is best, as will be covered in the sections that follow. Alternatively, you can simply switch up your tactics. For example, focus only on one memory operation function, such as `VirtualAlloc` OR `VirtualProtect`, rather than all of the functions listed in this chapter.

Second, breakpoints on memory can trigger unintended consequences during the malware's execution, usually in the form of exceptions and program crashes. Malware can take advantage of this by implementing special safeguards and evasion techniques in its code, as discussed in [Chapter 10](#); we'll revisit this briefly toward the end of this chapter.

Process Injection Monitoring

After unpacking their malicious code, malware packers must write this code into memory. This may involve writing it into its own process address space (self-injection) or injecting the code into a spawned child process or other victim process on the host. This is a critical part of how malware unpacks itself, and it's important to keep an eye out for. As mentioned previously, detonating the malware sample in a sandbox is always a good first step, but this is even truer when you're attempting to manually unpack the malware. Many sandboxes provide a great overview of how the malware is executing its process injection behaviors; such guidance will help you decide where to set breakpoints in the running malware sample while you're dynamically unpacking it.

To illustrate this, [Figure 17-23](#) shows the results of a suspect malware sample submitted to the Hybrid Analysis sandbox.

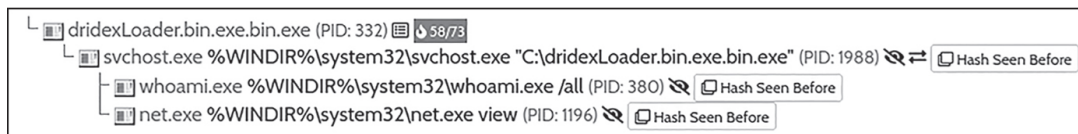


Figure 17-23: Malware sample behavior in the Hybrid Analysis sandbox

We can see that this malware sample is spawning a child process `svchost.exe`, which appears to be executed from the path

`C:\system32\svchost.exe`. This tells us that the malware sample may be unpacking its payload and injecting it into the legitimate `svchost.exe` Windows executable from the `system32` directory. If we wanted to unpack this malware sample manually by using a debugger, a good first step would be to set breakpoints on functions we predict would be invoked for this injection technique. For example, the malware would probably call `CreateProcess` OR `CreateProcessInternal` in order to execute the `svchost.exe` process. Next, it may invoke `WriteProcessMemory` to write the malicious code into the target process. And finally, in order to execute the malicious code in the context of `svchost.exe`, the malware may call a function such as `ResumeThread`. Setting breakpoints on each of these functions could help you catch the malware's payload in an unpacked state and extract it from memory for further analysis. All of these functions were described in [Chapter 12](#), so they shouldn't be completely new to you. Let's see what injection-based unpacking looks like in practice.

NOTE

To follow along with this example, you can find the required malware file on VirusTotal or MalShare using the following hash:

SHA256:

cfb959cc29e728cd0dc6d6f45bcd893fc91cad6f465720d63c5143001e63e705

The malware sample we're investigating, a variant of the Ryuk ransomware family, is using a process injection technique that involves getting a handle to a process (`OpenProcess`), allocating memory within that process (`VirtualAllocEx`), writing its unpacked code into the victim process (`WriteProcessMemory`), and finally executing this malicious code (`CreateRemoteThread`). `CreateRemoteThread` is a great function to investigate when you're unpacking malware because the code is fully unpacked at this point and about to be executed.

Load the malware sample into your debugger of choice (x64dbg, in my case) and set a breakpoint on `CreateRemoteThread`. Next, run the malware to hit the breakpoint on `CreateRemoteThread`, as shown in [Figure 17-24](#).

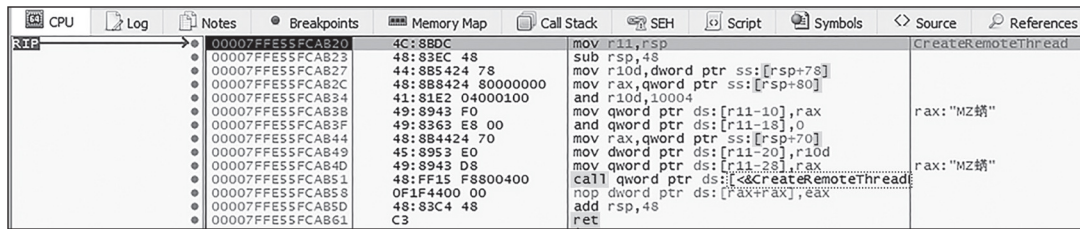


Figure 17-24: Malware calling `CreateRemoteThread` for process injection

`CreateRemoteThread` takes a few arguments, one of which is a handle to the process the malicious code was written to; this process is the one about to be executed. Since this malware sample is a 64-bit sample, this parameter for the `CreateRemoteThread` function is in the RCX register (see [Figure 17-25](#)). If this were a 32-bit sample, this value would be on the stack.

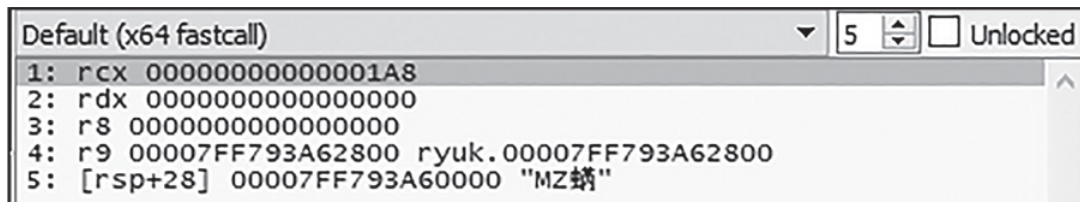


Figure 17-25: Investigating 64-bit CPU registers

In my case, this handle value is `0x1A8`. Cross-referencing this with the list of handles in the Handles tab of x64dbg, we can see that it is associated with process ID 2924, which, on my VM, is the system process `sihost.exe` (see [Figure 17-26](#)). Note that you may need to refresh the data in the handles tab by pressing F5.

Type	num	Handle	Access	Name
IRTimer	15	198	100002	
WaitCompletionPa	24	19C	1	
IRTimer	15	1A0	100002	
WaitCompletionPa	24	1A4	1	
Process	7	1A8	1FFFFF	PID: 2924 (\Device\HarddiskVolume1\Windows\System32\sihost.exe)
Token	5	180	20008	

Figure 17-26: A list of handles in x64dbg

To find the injected unpacked malicious code that's about to be executed, you need to locate the memory region where this code resides in the `sihost.exe` process. To do so, start another debugger instance and then attach to this process by selecting **File** ► **Attach** and choosing **sihost.exe** from the process list.

After attaching this process, you can use the Memory Map tab to locate the suspect region of memory. This suspect memory region will have ERW (Execute-Read-Write) protection and will be a private (PRV) memory region. **Figure 17-27** shows a screenshot of the suspect memory region.

00007FF5A5590000	00000000006A00000	Reserved (00007DF5ABF90000)		MAP		
00007FF6382A0000	00000000000001000	sihost.exe	Executable code	IMG	-R---	ERWC-
00007FF6382A1000	000000000000013000	".text"	Read-only initialized data	IMG	ER---	ERWC-
00007FF6382B4000	00000000000007000	".rdata"	Read-only initialized data	IMG	-R---	ERWC-
00007FF6382B8000	00000000000001000	".data"	Initialized data	IMG	-RW--	ERWC-
00007FF6382BC000	00000000000002000	".pdata"	Exception information	IMG	-R---	ERWC-
00007FF6382BE000	00000000000001000	".didat"		IMG	-R---	ERWC-
00007FF6382BF000	00000000000001000	".rsrsc"	Resources	IMG	-R---	ERWC-
00007FF6382C0000	00000000000001000	".reloc"	Base relocations	IMG	-R---	ERWC-
00007FF793A60000	00000000000389000			PRV	ERW--	ERW--
00007FFE40840000	00000000000001000	windows.staterepositoryps.dll	Executable code	IMG	-R---	ERWC-
00007FFE40841000	00000000000008000	".text"	Read-only initialized data	IMG	ER---	ERWC-
00007FFE40849000	000000000000119000	".rdata"		IMG	-R---	ERWC-

Figure 17-27: Suspected injected code in the memory map

To verify whether this memory region contains an unpacked executable, right-click it and select **Follow in Dump**. **Figure 17-28** shows that this memory region contains executable code!

Dump 1	Dump 2	Dump 3	Dump 4	Dump 5	Watch 1	[x=] Locals	Struct						
Address	Hex											ASCII	
0000000013F850000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00											MZ.....yy..	
0000000013F850010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00											@.....	
0000000013F850020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00												
0000000013F850030	00 00 00 00 00 00 00 00 00 00 00 00 F0 00 00 00											0...	
0000000013F850040	0E 1F BA 0E 00 84 09 CD 21 88 01 4C CD 21 54 68											...!.Li!Th	
0000000013F850050	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F											is program canno	
0000000013F850060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20											t be run in DOS	
0000000013F850070	6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00											mode...\$.	
0000000013F850080	33 28 0A 9E 77 49 64 CD 77 49 64 CD 77 49 64 CD											3(.wIdiwiIdiwi	
0000000013F850090	C3 D5 95 CD 70 49 64 CD C3 D5 97 CD 09 49 64 CD											A0.ipidiA0.i Idi	
0000000013F8500A0	C3 D5 96 CD 7A 49 64 CD 4C 17 67 CC 71 49 64 CD											A0.fzIdiL.gIqIdi	
0000000013F8500B0	4C 17 61 CC 55 49 64 CD 4C 17 60 CC 63 49 64 CD											L.aiUiDiL.iCIdi	
0000000013F8500C0	7E 31 F7 CD 70 49 64 CD 77 49 65 CD 03 49 64 CD											~i+ipIdiwiIdi	
0000000013F8500D0	E0 17 61 CC 75 49 64 CD E0 17 66 CC 76 49 64 CD											a.aiUiDiA.fivIdi	
0000000013F8500E0	52 69 63 68 77 49 64 CD 00 00 00 00 00 00 00 00											RichwIdi.....	
0000000013F8500F0	50 45 00 00 64 86 06 00 F8 E7 59 5E 00 00 00 00											PE..d...0cY^....	
0000000013F850100	00 00 00 00 F0 00 22 00 08 02 0E 00 00 14 01 00											0.".....	
0000000013F850110	00 D8 14 00 00 00 00 00 08 7B 00 00 00 10 00 00											.0.....{.....	

Figure 17-28: The unpacked executable in memory

Now, to confirm that this is indeed unpacked malicious code, let's dump this code to disk (in the Memory Map tab, right-click the target memory region and select **Dump Memory to File**) and open the file in a PE viewer such as PESTudio. As **Figure 17-29** shows, the Strings tab in PESTudio reveals some interesting things.

	encoding (2)	size (bytes)	file-offset	blacklist (68)	hint (55)	value (1958)
JA indicators (44)						
→ virustotal (error)						
→ dos-header (64 bytes)						
→ dos-stub (176 bytes)						
→ rich-header (10)						
→ file-header (Feb.2020)						
→ optional-header (GUI)						
→ directories (invalid)						
→ sections (9.26%)						
→ libraries (1)						
→ functions (n/a) *						
→ exports (n/a)						
→ tls-callbacks (n/a)						
→ .NET (n/a)						
→ resources (n/a)						
→ strings (size)						
→ debug (invalid)						
→ manifest (n/a)						
→ version (n/a)						
→ certificate (n/a)						
→ overlay (unknown)						
encoding (2)	ascii	7	0x00020D48	-	utility	encrypt
size (bytes)	ascii	12	0x00020D78	-	utility	explorer.exe
file-offset	ascii	6	0x00020D94	-	utility	sysprep
blacklist (68)	ascii	39	0x00020F78	-	utility	cmd.exe /c "WMIC.exe shadowcopy delete"
hint (55)	ascii	52	0x00020FA0	-	utility	cmd.exe /c "ssadmin.exe Delete Shadows /all /quiet"
value (1958)	ascii	47	0x00020FD8	-	utility	cmd.exe /c "bootstatuspolicy ignoreallfailures"
	ascii	79	0x00021030	-	utility	cmd.exe /c "bcdedit /set (default) recoveryenabled No & bcdedit /set (default)"
	ascii	8	0x00021080	-	utility	icacls
	unicode	7	0x000133C8	-	utility	encrypt
	unicode	12	0x000133F8	-	utility	explorer.exe
	unicode	6	0x000134A0	-	utility	Chrome
	unicode	4	0x000134C8	-	utility	IM
	unicode	7	0x0001D410	-	utility	DEL /F
	unicode	4	0x0001ED64	-	utility	calc
	unicode	4	0x0001FA96	-	utility	POP3
	unicode	4	0x0001FCBC	-	utility	SMTP
	unicode	8	0x00020100	-	utility	taskkill
	unicode	6	0x00020118	-	utility	stop
	unicode	90	0x000201F0	-	utility	/C REG DELETE "HKEY_CURRENT_USER\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"
	unicode	99	0x000202F0	-	utility	/C REG ADD "HKEY_CURRENT_USER\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"

Figure 17-29: Viewing strings from the unpacked code in PESTudio

Some of the incriminating strings include commands (specifically, *cmd.exe* commands that attempt to delete backup files from the host) and registry keys that may be related to establishing persistence on the host (*/C REG...*). When we compare these strings to the original malware sample file, we can see drastic differences (see [Figure 17-30](#)).

	encoding (2)	size (bytes)	file-offset	blacklist (19)	hint (20)	value (1953)
JA indicators (36)						
→ virustotal (error)						
→ dos-header (64 bytes)						
→ dos-stub (176 bytes)						
→ rich-header (10)						
→ file-header (Feb.2020)						
→ optional-header (GUI)						
→ directories (6)						
→ sections (99.25%)						
→ libraries (2) *						
→ functions (91) *						
→ exports (n/a)						
→ tls-callbacks (n/a)						
→ .NET (n/a)						
→ resources (n/a)						
→ strings (size)						
→ debug (PGO)						
encoding (2)	unicode	7	0x00011BC8	-	utility	encrypt
size (bytes)	unicode	12	0x00011BF8	-	utility	explorer.exe
file-offset	unicode	6	0x00011CA0	-	utility	Chrome
blacklist (19)	unicode	4	0x00011CC8	-	utility	IM
hint (20)	unicode	7	0x0001B010	-	utility	DEL /F
value (1953)	ascii	2666	0x0001B2C0	-	size	CqmKhGXRGSxmiIpoLiHmsgyDSsEtZtELzwGRKpnpfYRSIHceOowqwigCoeyPQeQcJKN...
	ascii	19	0x0001F750	-	rtti	AVbad_alloc@std@@
	ascii	19	0x0001F778	-	rtti	AVexception@std@@
	ascii	30	0x0001F7A0	-	rtti	AVbad_array_new_length@std@@
	ascii	15	0x0001F7D0	-	rtti	AVtype_info@@
	unicode	17	0x00011C30	-	privilege	SeBackupPrivilege
	ascii	12	0x0001A636	-	file	KERNEL32.dll
	ascii	10	0x0001A644	-	file	VS2_32.dll
	unicode	7	0x00011BA8	-	file	lan.exe
	unicode	9	0x00011BE0	-	file	csrss.exe
	unicode	9	0x00011C18	-	file	lsass.exe

Figure 17-30: Strings from the original, packed malware

Can you spot the differences in [Figure 17-30](#)? Many of the strings from the unpacked code are missing. As you can see, comparing the strings in the unpacked file with those in the original packed malware file can be an effective way to confirm that the malware was successfully unpacked.

Simply viewing strings has its limits, however. To further analyze the unpacked code, such as in a disassembler, we'd likely need to repair and realign the dumped code, which we'll cover shortly.

Process Injection Tracing with API Monitor

Tracing, which was introduced in [Chapter 3](#), is an effective technique not only for spying on malware's function calls but also for unpacking malware. [Figure 17-31](#) shows a malware sample in API Monitor. You can see the malware sample executing a process injection technique and using several functions you've seen in this chapter.

562	1:19:24.264 PM	1	clr.dll	CreateProcessA ("C:\Users\Public\evil.exe", "", NULL, NULL, FALSE, CREATE_NO_WINDOW CREATE_SUS...	TRUE
563	1:19:24.264 PM	1	KERNEL32.dll	↳ NtCreateUserProcess (0x0037cddc, 0x0037cdb8, MAXIMUM_ALLOWED, MAXIMUM_ALLOWED, NULL, 0...	STATUS_SUCCESS
564	1:19:24.279 PM	1	clr.dll	VirtualAllocEx (0x000002d4, 0x00400000, 167936, MEM_COMMIT MEM_RESERVE, PAGE_EXECUTE_REA...	0x00400000
565	1:19:24.279 PM	1	clr.dll	WriteProcessMemory (0x000002d4, 0x00400000, 0x043d0088, 512, 0x0037d704)	TRUE
566	1:19:24.279 PM	1	clr.dll	WriteProcessMemory (0x000002d4, 0x00401000, 0x043f84a8, 160256, 0x0037d704)	TRUE
567	1:19:24.279 PM	1	clr.dll	WriteProcessMemory (0x000002d4, 0x7efde008, 0x032c7b98, 4, 0x0037d704)	TRUE
568	1:19:24.279 PM	1	clr.dll	ResumeThread (0x000002d0)	1
569	1:19:24.279 PM	1	KERNELBASE.dll	↳ NtResumeThread (0x000002d0, 0x0037d38c)	STATUS_SUCCESS

Figure 17-31: Viewing process injection in API Monitor

After creating a new process (CreateProcessA) and allocating a new memory region in this process (VirtualAllocEx), the malware sample uses WriteProcessMemory to write code into this new memory region (see [Figure 17-32](#)). Selecting the WriteProcessMemory function in API Monitor and inspecting the Hex Buffer window reveals something interesting: an MZ header! This MZ header tells us that the malware wrote an executable to memory, and this can be copied and dumped from API Monitor.

Hex Buffer: 512 bytes (Post-Call)		
0000	4d 5a 45 52 e8 00 00 00 00 58 83 e8 09 8b c8 83 c0 3c 8b 00 03 c1 83 c0	MZER....X.....<.....
0018	28 03 08 ff e1 90 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	{.....
0030	00 00 00 00 00 00 00 00 00 00 00 00 b8 00 00 00 0e 1f ba 0e 00 b4 09 cd	
0048	21 b8 01 4c cd 21 54 68 69 73 20 70 72 6f 67 72 61 6d 20 63 61 6e 6e 6f	!..L!This program cannot
0060	74 20 62 65 20 72 75 6e 20 69 6e 20 44 4f 53 20 6d 6f 64 65 2e 0d 0d 0a	be run in DOS mode....
0078	24 00 00 00 00 00 00 00 7d 66 3f 1b 39 07 51 48 39 07 51 48 39 07 51 48	\$......}f?.9.QH9.QH9.QH
0090	22 9a fa 48 75 07 51 48 22 9a cf 48 3a 07 51 48 22 9a cc 48 38 07 51 48	".Hu.QH".H:.QH".H8.QH
00a8	52 69 63 68 39 07 51 48 00 00 00 00 00 00 00 00 50 45 00 00 4c 01 01 00	Rich9.QH.....PE..L...
00c0	ed 0c 71 3f 00 00 00 00 00 00 00 e0 00 02 01 0b 01 0a 00 00 72 02 00	..q?.....r..
00d8	00 00 00 00 00 00 00 00 c0 d0 01 00 00 10 00 00 00 90 02 00 00 00 40 00	@.
00f0	00 10 00 00 00 02 00 00 05 00 01 00 00 00 00 00 05 00 01 00 00 00 00	

Figure 17-32: Viewing WriteProcessMemory process injection in API Monitor

API tracing tools like API Monitor are excellent additions to your toolbox for following process injection and catching malware in the unpacking process.

Library Loading and Address Resolution

Earlier you learned that the packer's unpacking stub will likely use functions like `LoadLibrary` and `GetProcAddress` to dynamically resolve the malware's required functions. Because these functions are called before the malware can execute its malicious behaviors, `LoadLibrary` and `GetProcAddress` are great starting points for getting to the unpacked malware payload.

When you have the packed executable file loaded in a debugger, simply place a breakpoint on `LoadLibrary` and `GetProcAddress` and run the code by pressing F9. Optionally, place a breakpoint on `GetProcAddress` only, and you can skip all of the `LoadLibrary` operations. The first breakpoint to be hit will likely be `LoadLibrary`, which is responsible for importing the respective DLL library, and then `GetProcAddress` will get the address of the specific function exports from this DLL that the malware wishes to execute. If you continue to run the program until all function addresses have been resolved, this may be the point at which the malware is unpacked in memory and ready to start executing its malicious payload and functionalities.

From here, you can either dump the process from memory (which hopefully contains the unpacked executable) or try to locate the OEP.

OEP Location

Since all packers eventually transfer control flow to the malware's OEP after unpacking its payload, locating this OEP is one of the best and cleanest methods of unpacking. By "cleanest," I mean that dumping the unpacked malware at the OEP often results in an executable that most resembles the original, prepacked malware sample. Let's dig into how you can find the OEP.

Locating the Decryption Routine

First, to better understand the unpacking procedure that the malware will go through once executed, it's helpful to know how to locate decryption and decompression routines in the packed sample. [Chapter 16](#) described how to locate decryption routines in malware, and the process is mostly the same with packers. You can locate these routines in a debugger while debugging the packed sample or in a disassembler such as IDA Pro.

There are a few key indicators of possible decryption and decompression routines. First, there will likely be shift-related assembly instructions that are executed repeatedly, such as `xor`, `or`, `shl`, `shr`, and so on. You might also see many mathematical instructions, such as `add`, `sub`, `mul`, `imul`, and `div`. Finally, there will be looping, which indicates multiple iterations of decryption or decompression. Here's an example of what an unpacking routine in an unpacking stub may look like:

```
--snip--
movzx  edx, byte ptr [ecx]
shr    edx, 2
shl    esi, 6
lea    esi, [edx+esi+701h]
mov    edi, eax
sub    edi, esi
mov    dl, [edi]
mov    [eax], dl
mov    dl, [edi+1]
mov    [eax+1], dl
mov    dl, [edi+2]
mov    [eax+2], dl
inc    ecx
mov    [eax+2], dl
add    eax, 3
--snip--
```

In this malware sample code, you may have spotted the `shr`, `shl`, `sub`, and `add` instructions, which all hint that this code block is modifying data.

Also notice the move instructions (`movzx` and `mov`), which indicate that the code is shifting data around. Given these indicators, you can assume that this block of code may be loading encrypted (packed) data and decrypting (unpacking) it. But to be certain, locate the tailing jump, which we'll talk about now.

Finding the Tailing Jump

The tailing jump, as mentioned earlier in the chapter, occurs at the end of the unpacking stub and directs the malware's control flow to the newly unpacked code (more specifically, the OEP). Locating the tailing jump instruction can help you identify where the unpacked code will begin execution and is a good technique to use during the unpacking process. Since the tailing jump will be at the end of the unpacking procedure, it will likely come directly after the decompression and decryption routines you've just identified. Locating the tailing jump is easiest with a disassembler like IDA Pro. It should look something like **Figure 17-33**.

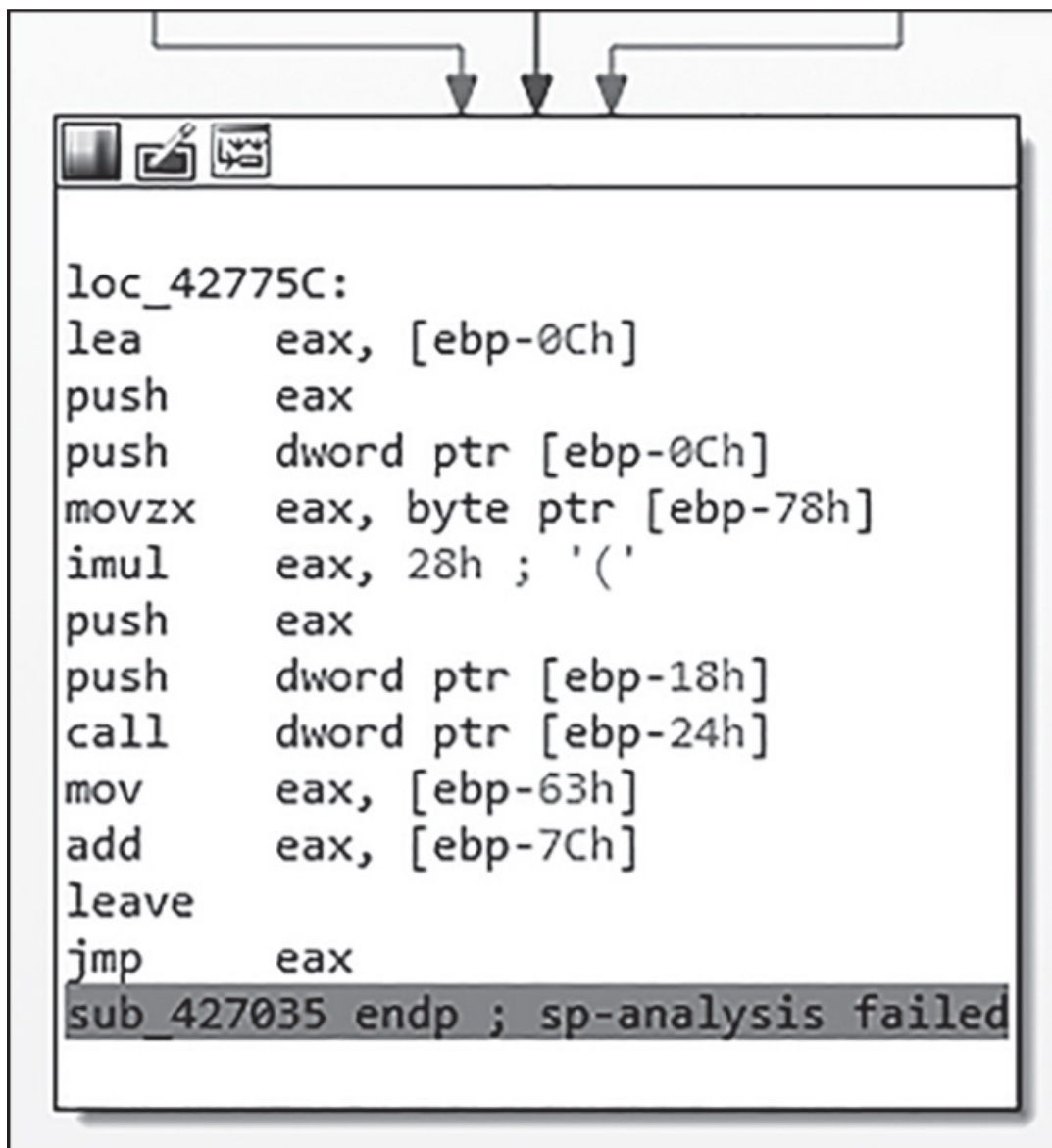


Figure 17-33: The packed malware's tailing jump in IDA Pro

In this screenshot, there are three arrows at the top of the code block, which means that other code blocks in the unpacking routine are jumping to this one. There is also a `call` instruction, followed by a `jmp` instruction. Both of these instructions reference dynamic locations (notice the CPU registers) rather than a static address, which is another good sign that one of these is the tailing jump. Since the malware could execute a `call` instruction instead of a `jmp` instruction, though, which one is actually the tailing jump? To answer this, you'd probably have to throw this sample into a debugger and set breakpoints on the addresses of these instructions to see what happens, or spend more time statically analyzing the unpacking stub's code. One of them leads to the OEP, the point at which the malware is unpacked and begins to execute its payload code.

NOTE

You may have noticed `sp-analysis` failed in the highlighted last line of the code in [Figure 17-33](#). This is IDA's way of stating that it couldn't disassemble the rest of this code, likely due to the fact that the rest of the code is dynamically resolved after unpacking and not available statically in the binary. This is another good indicator of packed code and the unpacking stub!

Finding the OEP Automatically

While the best method of locating the OEP is usually by first locating the tailing jump, there are also debugging tools and plug-ins that try to automatically locate the OEP in a sample. For example, OllyDbg (an older debugger that is still occasionally used by malware analysts and reverse engineers) has a built-in feature called SFX that may be useful in finding the unpacking routine and OEP. In OllyDbg, navigate to **Options ▶ Debugging Options ▶ SFX**, and you should see the options shown in [Figure 17-34](#).

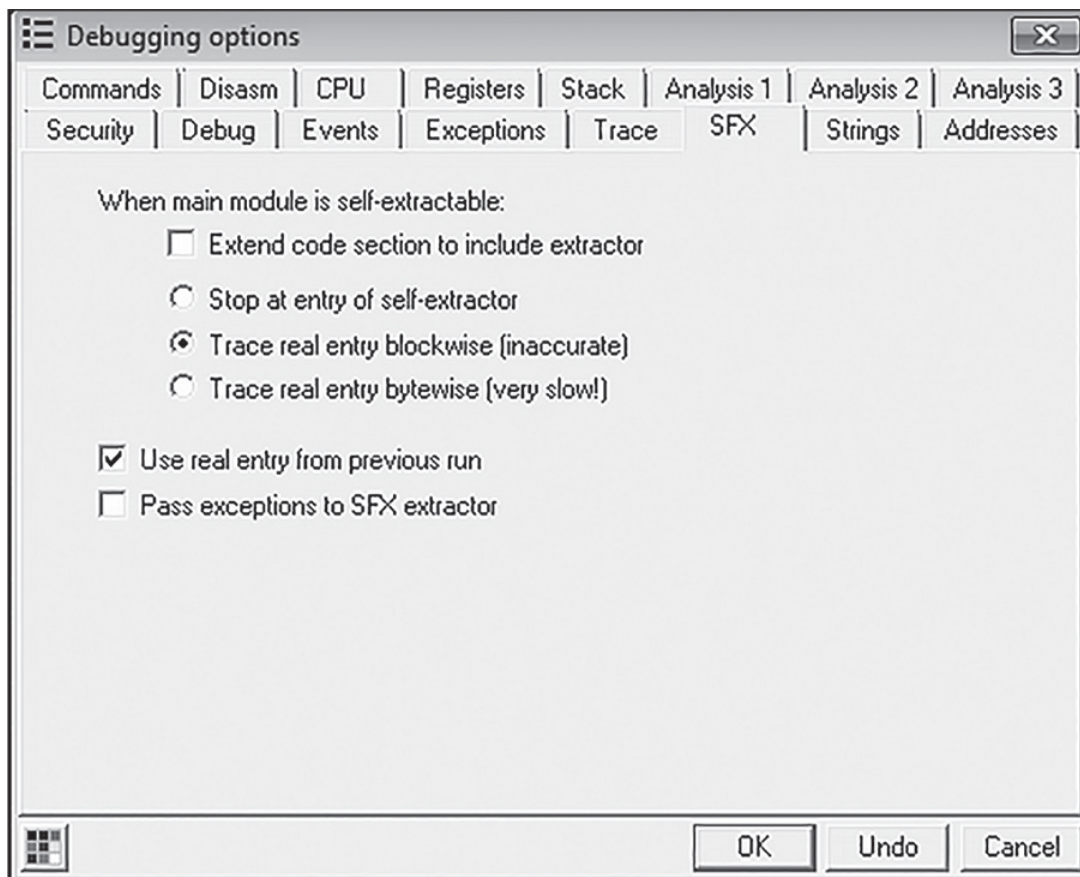


Figure 17-34: Debugging options in OllyDbg

Select either **Trace Real Entry Blockwise** or **Trace Real Entry Bytewise** and then run the executable. This tool will attempt to trace the unpacking code and break at the OEP.

Since tools for locating the OEP automatically are hit or miss, they likely won't be effective on advanced malware and packers. However, they're still options to keep in your toolbox and may save you some time and headaches.

Once you believe you've found the OEP (using any of the techniques discussed previously) and the sample is unpacked in memory, there are multiple ways of verifying that this is the true OEP and that the sample is fully unpacked. One approach is to inspect strings in memory. In x64dbg, simply right-click the mouse in the disassembler view and select **Search For ► Current Module ► String References**. An unpacked malware sample typically has human-readable strings loaded into memory that indicate some of its capabilities. If you see some suspect

strings in memory, the sample (or at least parts of it) is now unpacked in memory. In this window, you can also use the search bar at the bottom of the screen to search for a specific string.

You can also search for specific strings or binary patterns in memory by right-clicking in the disassembler window and then selecting **Search For ▶ Current Module ▶ Pattern**. This approach is useful if you know the malware has a certain capability that will be unpacked in memory. For example, if you know the malware will attempt to contact a C2 URL (such as *http://evil.com/c2.php*), you can specifically search for this pattern in the debugger; if it exists, the malware is likely unpacked or the string has otherwise been deobfuscated. Note that this could also mean simply that the string has been loaded onto the stack at this point in runtime.

Finally, you can inspect *intermodular calls*, the Windows API functions that the malware executes at some point, which may now be unpacked in memory. To do so, right-click in the disassembly view in x64dbg and select **Search For ▶ Current Module ▶ Intermodular Calls**. In an unpacked malware sample, there usually will be many interesting API functions listed here. Keep in mind, however, that some malware won't reveal its intermodular calls and will further unpack and resolve the functions at a later time.

Unpacked Malware Extraction

When you believe the malware's payload has been unpacked in memory, it's a good time to dump the unpacked code. Dumping the unpacked executable from memory will allow you to analyze the malicious code in more detail, such as in a disassembler. You can do this at any point in the unpacking process, so you don't necessarily need to have already located the OEP. However, finding the tailing jump and the subsequent control transfer to the OEP first is usually best. You also don't have to extract the payload from memory in order to analyze it further; you can

simply continue letting the malware run and analyze it in the debugger, as I'll discuss later in the chapter.

There are a few ways to dump the malware's unpacked payload from memory. One method is to use Scylla, and the steps are roughly the same as described earlier in the chapter. To launch Scylla in x64dbg, simply navigate to **Plugins ▶ Scylla**. Select the malware's process, and in the OEP field, enter the OEP address if you've located it or leave this set to the default OEP. You can also click **IAT Autosearch** and **Get Imports** to try to automatically rebuild the IAT. Finally, click **Dump**. After dumping the unpacked payload, you may also need to use the PE Rebuild option to repair the executable file.

Alternatively, you can use OllyDumpEx, which is included with some packages of x64dbg or can be found here: <https://github.com/x64dbg/x64dbg/wiki/Plugins>. OllyDumpEx allows more granularity and control but is a bit less user-friendly. To launch OllyDumpEx in x64dbg, navigate to **Plugins ▶ OllyDumpEx ▶ Dump Process**. OllyDumpEx allows you to select a process (module), a specific memory region, or a memory address to dump. Which option you use depends on how the sample has been unpacked and whether you've successfully found the OEP. If you've found the unpacked malware's OEP, select the malware process module and click **Get EIP as OEP** and then **Dump**, as shown in [Figure 17-35](#).

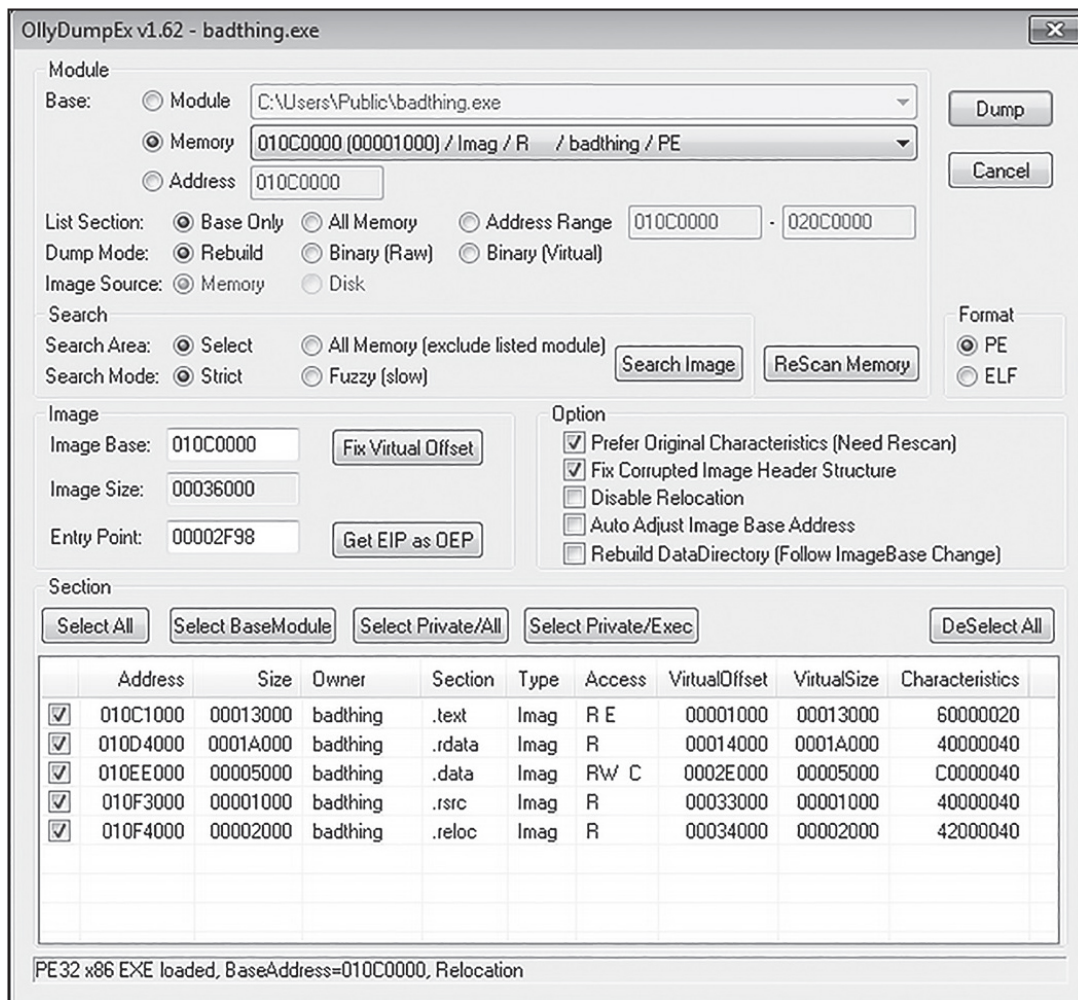


Figure 17-35: The OllyDumpEx plug-in in x64dbg

Lastly, you can use the native x64dbg interface to save the suspect memory region to disk. Right-click the disassembler window and select **Follow in Memory Map**. Next, select the region of memory that contains the unpacked code, right-click it, and select **Dump Memory to File**.

In this case, the extracted memory will be in the form of binary data, usually saved as a *.bin* file. Running a strings utility (such as Strings or PE Viewer) on this extracted memory is always a good first step. Looking at the strings can provide clues about how to proceed with your analysis. If you load this file into a disassembler to view its code, however, you might be disappointed to find it needs some repairs.

Unpacked Executable Repair

Once the unpacked payload has been dumped to disk, depending on how it was extracted, it may not be in a runnable state or cleanly analyzable in a disassembler. Common reasons for this are file misalignment, PE header corruption, or a broken IAT, all of which can also make static analysis of the executable problematic. Let's look at how to fix these issues.

Automated Repair

Scylla will allow you to rebuild a broken executable dump file. After you dump the malware's suspected unpacked payload from memory, launch Scylla (either the stand-alone version or the plug-in built into x64dbg), select the malware's running process, enter the correct OEP (if you have identified it), and click **IAT Autosearch** and then **Get Imports**. You may get a message asking whether or not to use the advanced results. Select **No** for now. (Try the advanced results later if this method doesn't work correctly.) Next, click **Fix Dump**, select your memory dump, and save the file. After inspecting the newly generated executable, if the PE headers are corrupted, you can also click **PE Rebuild** to rebuild the headers. There are other tools you could use to rebuild the IAT, such as Imports Fixer, ImpREC, and ChimpREC, but I have found Scylla to be one of the best for imports reconstruction. Feel free to test out these other tools if Scylla isn't working in your specific situation.

Another one of my go-to tools for fixing unmapped (dumped) PE executables is PE Unmapper (https://github.com/hasherezade/pe_unmapper). This tool can help automate the process of unmapping an executable image from memory into a file and realigning the executable. Here's an example of it in use:

```
C:\> pe_unmapper.exe dumped_executable.mem 0x13F630000 fixed_executable.exe
```

To execute PE Unmapper, you point the tool (`pe_unmapper.exe`) at your target memory image (`dumped_executable.mem`, in this example), specify the base address of the image (where you dumped the image from memory; in this case, `0x13F630000`), and then specify the output filename (in this case, `fixed_executable.exe`).

Manual Realignment

If your automated tools fail, you’ll need to manually realign the headers using a PE editor tool such as PE-bear. In PE-bear, simply load the executable file and navigate to the **Section Hdrs** tab, and you should see all sections and their associated raw and virtual sizes. Remember, the raw size represents the size of the file on disk, and the virtual size represents the size once it is loaded into memory. Since we unmapped the file from memory, we need to get the file on disk to match the file that was in memory as closely as possible. [Figure 17-36](#) shows what the dumped executable may look like prior to section realignment.

Disasm: .text	General	DOS Hdr	Rich Hdr	File Hdr	Optional Hdr	Section Hdrs	Imports
+ [icon]							
Name	Raw Addr.	Raw size	Virtual Addr.	Virtual Size	Characteristics	Ptr to Reloc.	Num. of
▷ .text	400	11400	1000	11400	60000020	0	0
▷ .rdata	11800	9400	13000	9292	40000040	0	0
▷ .data	1AC00	4C00	1D000	1428B0	C0000040	0	0
▷ .pdata	1F800	1000	160000	ED0	40000040	0	0
▷ .gfids	20800	200	161000	BC	40000040	0	0
▷ .reloc	20A00	800	162000	638	42000040	0	0

Figure 17-36: Viewing the newly unmapped executable in PE-bear

Notice how the raw (Raw Addr.) and virtual (Virtual Addr.) address offsets do not match. To fix and realign the file, first ensure each raw address matches its associated virtual address. This can be a simple copy-paste operation from the Virtual Addr. column to the Raw Addr. column. For example, if the virtual address of the `.text` section is `1000`, then the raw address should be `1000` as well.

Next, we need to recalculate the raw size to match the new raw addresses in order to allocate space in the file. Subtract the raw address of

the first section (usually `.text`) from the raw address of the next listed section (usually `.rdata`). For example, if the raw address of the `.text` section is 1,000 bytes and the raw address of `.rdata` is 13,000 bytes, the raw size of the `.text` section should be `A000` (12,000 bytes in decimal). You'll need to do this for each address. For the last section, you can try entering 0 bytes, which will usually be okay. If this doesn't work, try changing it to something like 1,000 bytes. [Figure 17-37](#) shows what the file should look like afterward.

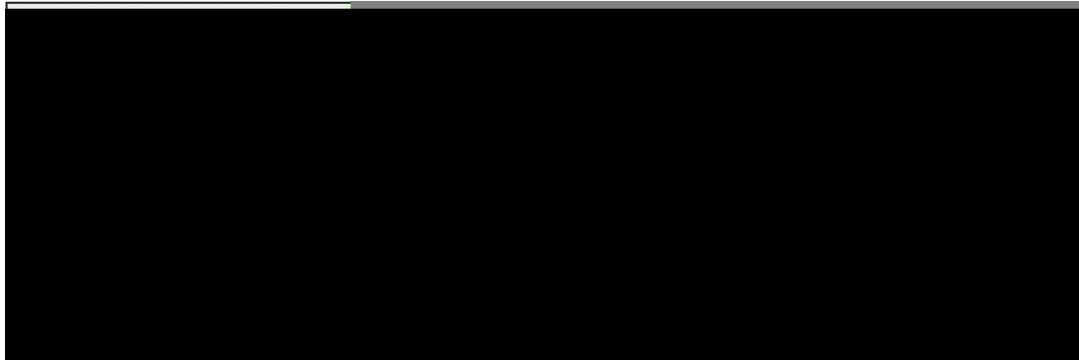


Figure 17-37: Viewing the fixed executable in PE-bear

To test whether the realignment was successful, navigate to the **Imports** tab in PE-bear and you should see the imports listed. If needed, you can use Scylla to attempt to rebuild the IAT and headers as described previously.

If automated and manual repairing are ineffective and the unpacked executable still won't run, you may not necessarily need to repair it. You could simply continue to examine the malware in a debugger (since the unpacked payload was already about to execute when you dumped it from memory), or you could try to analyze the unpacked executable as it is in a disassembler or PE tool. It just may be a bit more difficult to navigate, and you may need to manually label Windows function calls.

General Tips for Dynamic Unpacking

Sometimes these techniques just won't cut it. The malware could be using an uncommon unpacking method, or maybe it's especially stealthy

in the way it unpacks or injects its code. If you're in this situation and have hit a wall, there are some general tips that may help you get unstuck.

Working Backward

If you are analyzing a stubborn malware sample and are unable to follow the unpacking process or locate the OEP, working backward can help.

First, you identify a certain behavior that you know the malware is exhibiting and determine which Windows API function is likely responsible for this behavior. For example, if the malware attempts to contact a C2 address via HTTP, it might call the function `InternetConnectA` after its payload is unpacked. Or, if the malware is creating and modifying files on the disk, it might call `WriteFile`.

Next, set a breakpoint on the functions you've identified. Once the breakpoint is hit, walk backward in the code and try to find the unpacking routine, the memory region where the sample first unpacked itself, or (even better) the OEP.

Hooking Windows Decryption and Compression Functions

Malware packers may call native Windows API decryption-related functions during the unpacking process. Two of these functions are `CryptDecrypt` and `RtlDecompressBuffer`. `CryptDecrypt` is used to decrypt data that was previously encrypted with the `CryptEncrypt` function. Setting a breakpoint on `CryptDecrypt` may enable you to catch part of the malware's unpacked payload (or other juicy data) directly after it has been decrypted and before it is executed. After the breakpoint has been hit, examine the buffer passed into the `CryptDecrypt` function (usually the *fifth* value on the stack) after the function call.

`RtlDecompressBuffer` is sometimes used by malware to decompress a buffer that was previously compressed. As with `CryptDecrypt`, set a break-

point on `RtlDecompressBuffer` and examine the buffer (usually the *second* value on the stack) after the function call. You may get lucky and see newly unpacked code or an executable in this buffer.

Locating Packed Code

IDA Pro and some other disassemblers have a feature that visually represents the data and code in an executable file as colored boxes. You can find this visual at the top of the IDA interface, as shown in **Figure 17-38**.

Figure 17-38: A visual representation of a file in IDA Pro

Although you can't see it in a black-and-white book, this visual representation assigns different colors to the different types of data contained in the file. The Unexplored section (at the far-right of this image) is an area of the executable where IDA can't determine the type of data. Sometimes these regions are encrypted or packed data or code, or they are areas where the malware will write data once it is executed. These regions are typically assigned a name (such as `unk_4141C0` or `dword_1000502C`). If you select the region name and press X (a shortcut for cross-reference), you should see a list of the areas of code that reference that region. Exploring these code areas may lead you to the malware's main unpacking routine! Alternatively, setting a breakpoint on a code area referencing the unexplored region may give you some insight into how the malware is using the region.

Helpful Tools for Dynamic Unpacking

Because unpacking can be very difficult at times, it is helpful to have a set of tools that you can rely on when the going gets tough. This section will outline some of my favorites.

HollowsHunter

HollowsHunter (https://github.com/hasherezade/hollows_hunter), which is built on a tool called PE-Sieve (<https://github.com/hasherezade/pe-sieve>), is a Windows command line tool that can detect various anomalies in running processes (such as injected PE files and code, hooked functions, and in-memory patches) and then dump the suspect regions of memory. While it’s not a tool solely for unpacking, it’s also great at that. **Table 17-2** lists some of the most useful HollowsHunter parameters.

Table 17-2: Helpful HollowsHunter Parameters

Parameter	Description
/help	Shows all commands and their usage.
/pid <i>pid</i>	Specifies a target process ID to scan, rather than scanning all processes. Can also specify multiple target process IDs.
/loop	Continues to loop after the initial scan is completed. Good for monitoring the running processes on a system in case there is a delay in malware unpacking or code injection.
/data	Scans nonexecutable memory regions as well as executable regions. Enable this if you suspect the malware may be writing code or data and setting it to a nonexecutable protection (R, W, RW, and so on).
/hooks	Scans for memory patches and inline hooks.
/iat <i>mode</i>	Scans for IAT hooks. Setting <i>mode</i> to 1 will produce a filtered scan, leaving out the noise of system IAT hooks.

Parameter	Description
<code>/shellc</code>	Scans for shellcode injections. Can be a bit noisy, so use with caution.
<code>/imp mode</code>	Attempts to recover the imports table of any dumped executables. Setting <i>mode</i> to 1 will attempt to automatically detect the correct method of imports reconstruction.

To use HollowsHunter, execute the malware in your analysis environment and run HollowsHunter with your desired command line options, such as the following:

```
C:\> hollows_hunter64.exe /loop /hooks /shellc /iat 1 /imp 1
```

This command tells HollowsHunter to continue to loop through all the running processes on the system (`/loop`), specifically searching for hooks (`/hooks`), injected shellcode (`/shellc`), and IAT hooks (`/iat 1`). Finally, HollowsHunter will attempt to rebuild the IAT (`/imp 1`) of the dumped unpacked executable files.

After you run HollowsHunter, it will attempt to detect malicious code in memory. For example, in [Figure 17-39](#), HollowsHunter has detected potentially malicious code in the *RuntimeBroker.exe* and *dllhost.exe* processes. This code could be the result of the malware unpacking itself and subsequently injecting code into these processes.

Figure 17-39: HollowsHunter scanning process memory

Once it detects suspicious code, HollowsHunter dumps the suspect memory regions to disk and nicely organizes all the dumped memory images into a series of directories by process ID, as shown in [Figure 17-40](#).

Figure 17-40: The HollowsHunter output

HollowsHunter is often one of the first unpacking techniques I try. Sometimes I just need to get to the unpacked malware sample quickly, without messing around with the unpacking process. A similar tool, Mal_Unpack (https://github.com/hasherezade/mal_unpack), is written by the same author and can also be very helpful. Like HollowsHunter, Mal_Unpack is based on PE-Sieve, but it uses a special optional driver that allows for better control of the automated unpacking process. **Figure 17-41** shows Mal_Unpack in action.

Figure 17-41: Mal_Unpack unpacking a Dridex malware sample

As you can see, Mal_Unpack was able to unpack a malware sample that is part of the Dridex family!

Both tools can be valuable additions to your analysis arsenal. However, since HollowsHunter is able to scan all processes running on the host, it can often identify and locate code that malware has injected into other processes. For this reason, I often find that it suits my needs better. I recommend trying out both tools to determine which works best for you.

ScyllaHide RunPE Unpacker

ScyllaHide, which I've mentioned several times throughout this book, has an option called RunPE Unpacker that attempts to automatically extract executable files from memory when it detects that unpacking has occurred. The feature hooks `NtResumeThread` to intercept certain process injection techniques and dump the unpacked malware before it can execute its payload.

To use this feature, load your malware sample into x64dbg and select **Plugins ▶ ScyllaHide ▶ Options**. Check **RunPE Unpacker** and click

Apply, as shown in **Figure 17-42**.

Figure 17-42: RunPE Unpacker in ScyllaHide

After enabling this feature, run the malware as normal. If the unpacking is successful, you should see a newly created executable file on your desktop; you won't receive any other notification about whether or not it is successful. RunPE Unpacker targets a specific process injection technique and does not work in all cases. When it does work, however, it saves you a lot of time.

A Note on Emulation and Instrumentation

Emulators and binary instrumentation frameworks, which we'll briefly discuss in **Appendix A** can also be useful for dynamically unpacking malware. These toolsets can even provide a means of completely automating the unpacking process. For example, Speakeasy (<https://github.com/mandiant/speakeasy>) allows you to emulate malicious code and "hook" suspect function calls. As Speakeasy operations can be scripted, it's possible to automate the interception of function calls such as `VirtualProtect`, `WriteProcessMemory`, or `ResumeThread` in order to automate malware unpacking. I won't touch on emulators or instrumentation more in this chapter, but know that many of the techniques you learned in this chapter can be applied to these toolsets.

Other Tools

Finally, you can find several scripts and plug-ins for x64dbg that can aid in unpacking on GitHub at <https://github.com/x64dbg/Scripts> and <https://github.com/x64dbg/x64dbg/wiki/Plugins>. These scripts and plug-ins can automatically unpack certain packers, automatically locate the OEP, hide your debugger from unpackers, help bypass anti-analysis checks, and more.

As a final note, there are always new and innovative research projects and tools being released from the amazing malware research and reverse engineering community, so there's no way I can capture them all here. Always be on the lookout for newly released tools that can aid in unpacking and in malware analysis more generally.

Manual Static Unpacking

The final method of unpacking we'll discuss is *static unpacking*, the process of reverse engineering the unpacking mechanism in a packed malware sample and then writing code that replicates that mechanism. Once this code is run on the packed malware executable, the malware sample will theoretically be unpacked. The process of writing a static unpacker is roughly as follows:

1. Locate the unpacking routine in the packed malware executable.
2. Locate the specific instructions that are part of the deobfuscation or decryption process.
3. Reverse engineer the decryption routine (this can be done with the help of a decompiler, such as the ones built into IDA Pro, Ghidra, or x64dbg).
4. Write the static unpacking code to model the unpacking routine in the executable.
5. Feed the packed malware sample into your static unpacker and test it out! Debug as necessary.

I chose not to go into depth on this unpacking technique for two reasons. First, this process can be very time-consuming and tedious, and learning about other unpacking techniques may be a better use of your time, assuming your goal is to quickly unpack the malware sample so that you can better understand it. Second, if you were to successfully reverse engineer the unpacking technique and write a static unpacker,

your code might only work properly on this particular sample, as malware packers often introduce some sort of randomness to the unpacking routine (a random decryption key, obfuscation technique, or something else). Static unpackers are not resilient to heavy modification of the packer's code.

This doesn't mean that there's no value in manual static unpacking. Static unpackers are often better for at-scale analysis. For example, if you're attempting to unpack tens or hundreds of samples at a time, it's more efficient to do so statically than to run all those samples through a dynamic engine. Also, reversing the unpacking process and writing an unpacker is a great way to learn how malware packs its code and, more generally, a great way to learn about encryption, compression, and obfuscation.

Analyzing Without Unpacking

Sometimes you may find yourself unable to successfully unpack a malware sample. Maybe you lose control of the sample while it is running, can't determine where and how the malware is unpacking itself, or can't find the OEP. When in this situation, ask yourself if unpacking the malware is really necessary. What are you trying to achieve by doing so? What questions are you trying to answer?

In many cases, you don't have to fully unpack the sample to understand its key behaviors or even to perform code analysis. You may be able to extract pieces of the unpacked malware sample from memory using the techniques described throughout this book. This will allow you to at least perform some analysis of the data, code, and extracted strings. Alternatively, you can inspect the running malware in a debugger, monitor its behaviors, set breakpoints on interesting function calls, and examine code and strings in memory. Finally, sometimes simply examining the malware executing in an automated sandbox is all you need to be able to understand the malware's basic functionalities.

Anti-unpacking Techniques

Up until now, we've been discussing typical methods that malware packers use to decrypt and deobfuscate the malware's payload, write it into memory, and execute it, sometimes in a covert manner. We've also talked about common methods of automatically and manually unpacking the malware so that it can be better analyzed and understood. But what if the malware packer itself fights back and attempts to evade your tools and analysis? Some packers, especially those designed specifically for malware, implement some form of VM and sandbox detection techniques and attempt to evade the analysis process in order to protect the malware's original code. For this reason, it's important to know and understand common anti-unpacking techniques used by advanced packers.

Many of these techniques have already been discussed throughout this book. For example, malware packers often implement the sandbox and VM detection techniques covered in [Part II](#). Before unpacking the malicious code, the packers try to identify whether the sample is running in a VM or sandbox environment and, if so, whether it will avoid unpacking.

Many malware packers also take advantage of the techniques discussed in [Chapter 10](#) to detect and obstruct debugging programs. For example, before unpacking and running the malicious code, the packer tries to detect if it is being debugged and kills itself if it is. It might also attempt to interfere with the manual unpacking process by using anti-debugging techniques, such as interfering with breakpoints or using memory guard pages. Some packers even implement the sandbox evasion techniques discussed in [Chapter 8](#). For example, they may sleep for a certain amount of time before unpacking and executing the malware's payload in order to create a timeout situation in malware sandboxes and confuse the analysis process.

Basic anti-unpacking techniques can be as simple as modifying the original packer so that it's more difficult to detect and unpack, which can be

accomplished with any well-known or open source packer. The malware author could pack their sample with the UPX packer, for example, but modify it to remove the normal section name strings (UPX0, UPX1, and so on) or corrupt its headers. Then, when the malware analyst tries to identify the packer, they won't be able to detect UPX as easily. Additionally, the packer's code could be modified so that the normal UPX unpacking is impossible (by using the UPX tool, for example). Packers can also modify or destroy the unpacked executable's headers so that when the sample is unpacked, it's more difficult for automated tooling and malware analysts to locate the unpacked PE.

IAT confusion is another technique employed by malware packers. After the packer has resolved the initial imports and functions, it may alter or completely destroy the IAT and then rebuild it at a later point by dynamically resolving function addresses. Similarly, the packer may allocate a separate area of memory, like a jump table, to store instructions that jump to the functions it wishes to execute as a way of obfuscating its IAT.

Finally, malware packers can obfuscate themselves through sheer confusion. It's not uncommon for the packer to have a very convoluted control flow, with spaghetti code and control flow transfers (such as some of the techniques discussed in [Chapters 9](#) and [11](#)) that make following the code and unpacking process difficult. The packer might also unpack the malicious code in multiple steps, further obfuscating the unpacking process and making it difficult for analysts to cleanly unpack the malware's payload.

VIRTUAL MACHINES IN PACKERS

The use of packer virtual machines is an anti-unpacking and software protection technique not only in malware but also in legitimate software. Some of the more advanced packers use a small VM included in the packed executable. The packer essentially transforms parts of the original executable code in the packed binary into a proprietary intermediate language that exe-

cutes in the mini-VM. This creates a scenario in which the original executable's code isn't clearly visible in memory. You can probably guess why this makes a reverse engineer's life a lot more difficult.

Perhaps the best-known packers that utilize this technique are VMProtect and Themida. Since these packers are commercial and used mostly for protecting legitimate software, it is rare for malware to use them. There are a few reasons for this. First, it's fairly trivial to detect malware packed with them because they weren't designed with anti-malware and detection evasion in mind. Second, it can be difficult to use these packers correctly, and any misconfigurations can render them ineffective. And third, virtualized packers increase the file size of the malware and are slower than running the executable natively.

Because these types of packers aren't often seen in the wild in malware, I don't say much more about them in this book. If you spot a malware sample using one of them, remember that the malware author is human, and humans make mistakes. Virtualized packers are more difficult to use, so misconfigurations can occur. If the packer is misconfigured, you might be able to unpack the malware sample using the techniques discussed throughout this chapter, bypassing the virtualization completely.

Summary

In this chapter, you learned about some common packer types and how malware can leverage packers to obfuscate code, obstruct analysts and researchers from understanding its behaviors, and evade detection and defensive tools. You also saw several techniques you can use to peel back the unpacker's layers and get to the source of the malware's evil: its payload. Finally, we briefly examined some ways in which malware packers may try to circumvent analysis measures.

In **Appendix A**, we'll look at how to build an effective anti-evasion analysis lab, which can be a great asset in your evasive-malware

investigations.

Closing Thoughts

This chapter marks the end of *Evasive Malware*. I hope you've obtained new skills you can start applying to your threat investigations and analysis efforts. But above all, I hope this book has deepened your interest in the area of evasive threats and given you a thirst for more knowledge.

I encourage you to continue researching this fascinating topic. A great start would be to consult [Appendix C](#), which contains some recommended resources and further reading. Then, test out your new skills. Take apart an interesting piece of malware. Experiment and uncover how it attempts to evade defenses and your analysis tooling. And most importantly: document your findings for others! We are much more powerful fighting the battle against malware and cybercrime together than we are individually. Thank you for reading.